


# Data Mining and Big data anlysis

# 数据挖掘及大数据分析

## Chapter 6：Mining Frequent Patterns, Associations, and Correlations

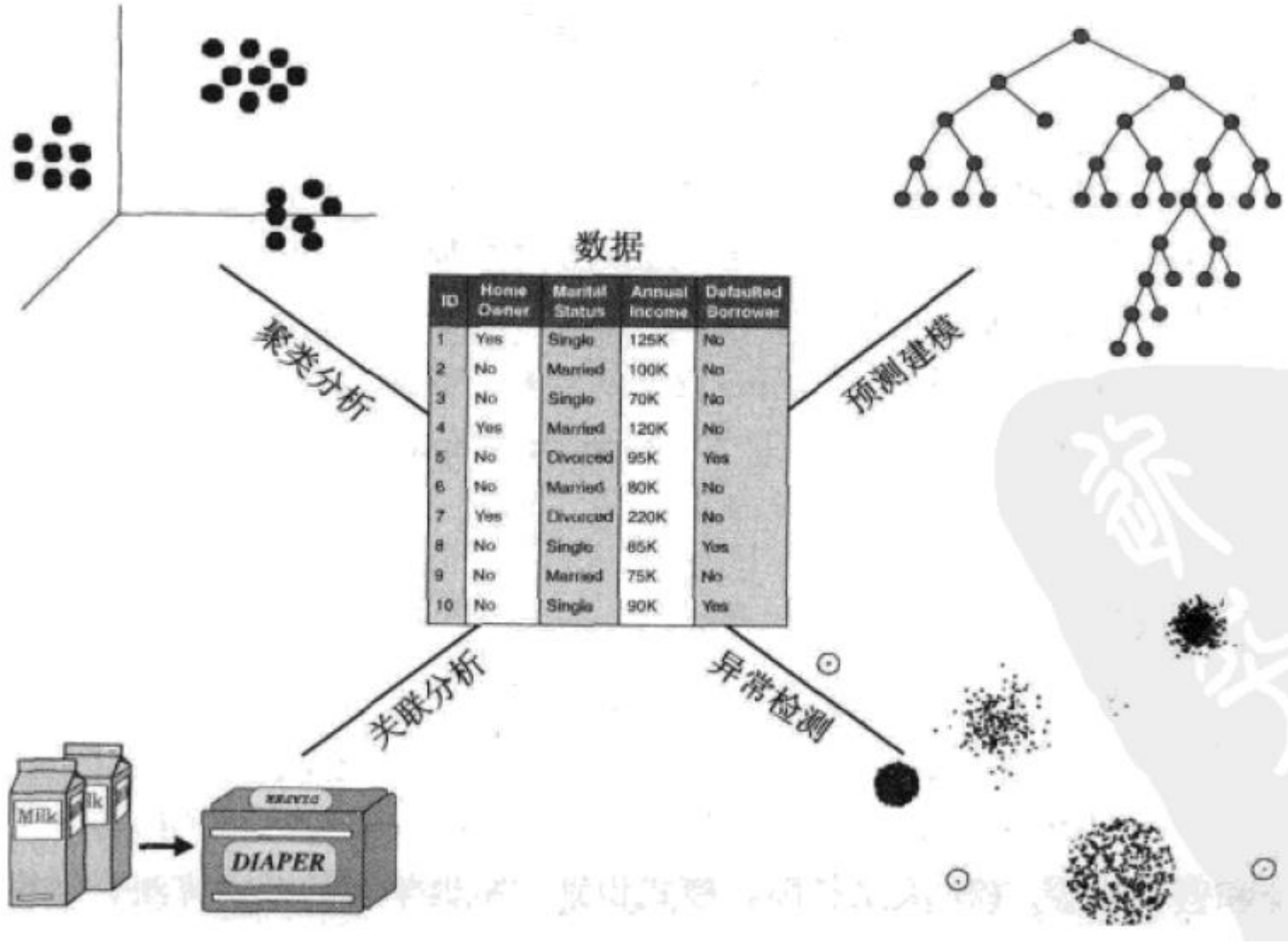
## 挖掘频繁模式、关联和相关性：基本概念和方法

王东、张谦、丁玥

上海交通大学软件学院RFID与物联网实验室

2025年春季

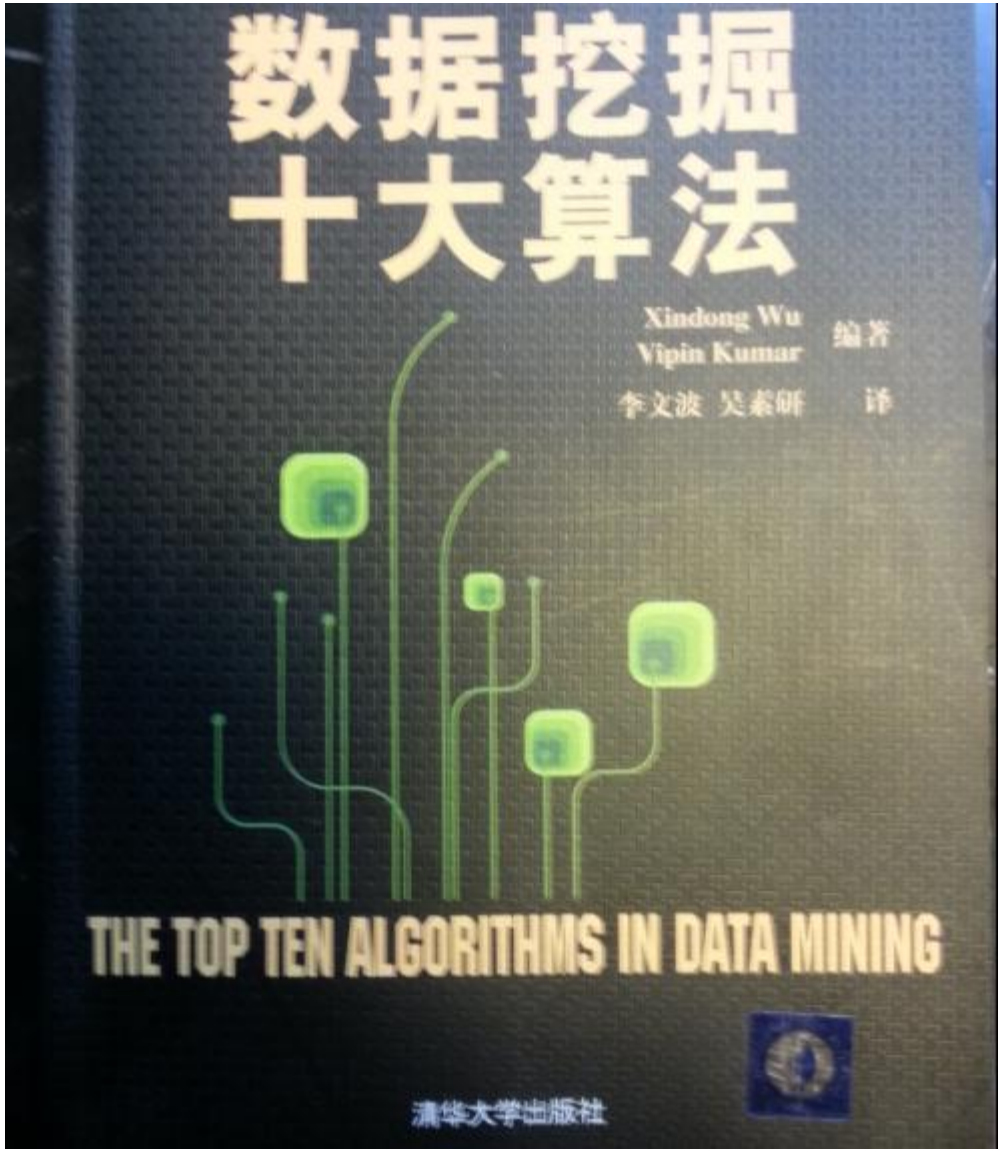
# Data Mining Function
# 4种经典数据挖掘模式



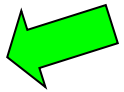

| ID | Home Owner | Marital Status | Annual Income | Defaulted Borrower |
|---|---|---|---|---|
| 1 | Yes | Single | 125K | No |
| 2 | No | Married | 100K | No |
| 3 | No | Single | 70K | No |
| 4 | Yes | Married | 120K | No |
| 5 | No | Divorced | 95K | Yes |
| 6 | No | Married | 80K | No |
| 7 | Yes | Divorced | 220K | No |
| 8 | No | Single | 85K | Yes |
| 9 | No | Married | 75K | No |
| 10 | No | Single | 90K | Yes |

# 数据挖掘的“十大”最具影响力的经典算法

- C4.5
- k-means
- SVM：支持向量机
- Apriori
- EM：期望最大化
- PageRank
- AdaBoost
- KNN：k-最近邻
- Naïve Bayes
- CART：分类和回归树



# Chapter 6
# 挖掘频繁模式、关联和相关性基本概念和方法

- 基本概念 
- 频繁项集挖掘方法
- 哪些模式是有趣的：模式评估方法
- 小结

# 什么是频繁模式分析?

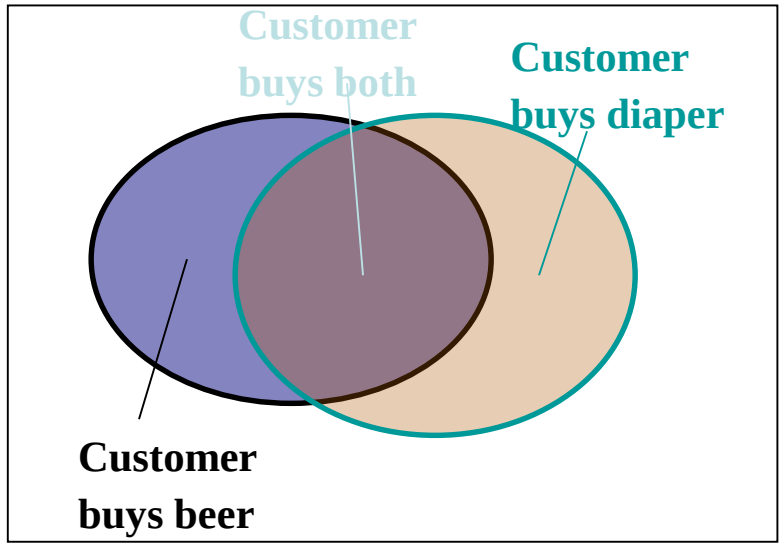
- 频繁模式Frequent pattern: 是频繁地出现在数据集中的模式（如项集、子序列或子结构）

- 动机: 寻找数据中的内在规律
  - 经常在一起购买什么样的产品- 啤酒和尿布？！
  - 什么是购买一台PC以后购买的？
  - 什么样的DNA对这个新的药物敏感？

- 应用
  - 购物篮数据分析,跨界营销,目录设计,销售活动分析, Web日志（点击流）分析, DNA序列分析

# 支持度和置信度

条件概率是事件B在另外一个事件A已经发生条件下发生概率。条件概率表示：P（B|A），读作“在A的条件下B的概率”




| TID | 购买的item |
|---|---|
| 2000 | A,B,C |
| 1000 | A,C |
| 4000 | A,D |
| 5000 | B,E,F |

- 支持度s是指事务集D中包含的百分比 $A \cup B$

$$\sup port(A \Rightarrow B) = P(A \cup B)$$

- 置信度c是指D中包含A的事务同时也包含B的百分比

$$confidence\,(A \Rightarrow B) = P(B \mid A) = P(A \cup B) / P(A)$$

- 假设最小支持度阈值为50%，最小置信度阈值为50%，则有如下关联规则
  - $A \Rightarrow C$ (50%, 66.6%)
  - $C \Rightarrow A$ (50%, 100%)
- 同时满足最小支持度阈值和最小置信度阈值的规则称作强规则

# Definition: Frequent Itemset

- Itemset
  - A collection of one or more items
    - Example: {Milk, Bread, Diaper}
  - k-itemset
    - An itemset that contains k items
- Support count (σ)
  - Frequency of occurrence of an itemset
  - E.g. σ({Milk, Bread,Diaper}) = 2
- Support
  - Fraction of transactions that contain an itemset
  - E.g. s({Milk, Bread, Diaper}) = 2/5
- Frequent Itemset
  - An itemset whose support is greater than or equal to a *minsup* threshold

| *TID* | *Items* |
|---|---|
| 1 | Bread, Milk |
| 2 | Bread, Diaper, Beer, Eggs |
| 3 | Milk, Diaper, Beer, Coke |
| 4 | Bread, Milk, Diaper, Beer |
| 5 | Bread, Milk, Diaper, Coke |

$$s = \frac{\sigma(\text{Milk, Diaper, Beer})}{|T|} = \frac{2}{5} = 0.4$$

Diaper [daɪpərn] 尿布

# Definition: Association Rule

- Association Rule
  - An implication expression of the form X → Y, where X and Y are itemsets
  - Example:
    {Milk, Diaper} → {Beer}

- Rule Evaluation Metrics
  - Support (s)
    - Fraction of transactions that contain both X and Y
  - Confidence (c)
    - Measures how often items in Y appear in transactions that contain X

{Milk,Diaper} → {Beer} (s=0.4)
{Milk,Beer} → {Diaper} (s=0.4)
{Diaper,Beer} → {Milk} (s=0.4)
{Beer} → {Milk,Diaper} (s=0.4)
{Diaper} → {Milk,Beer} (s=0.4)
{Milk} → {Diaper,Beer} (s=0.4)

| TID | Items |
|---|---|
| 1 | Bread, Milk |
| 2 | Bread, Diaper, Beer, Eggs |
| 3 | Milk, Diaper, Beer, Coke |
| 4 | Bread, Milk, Diaper, Beer |
| 5 | Bread, Milk, Diaper, Coke |

Example:

$$\{\text{Milk, Diaper}\} \Rightarrow \text{Beer}$$

$$s = \frac{\sigma(\text{Milk, Diaper, Beer})}{|\text{T}|} = \frac{2}{5} = 0.4$$

$$c = \frac{\sigma(\text{Milk, Diaper, Beer})}{\sigma(\text{Milk, Diaper})} = \frac{2}{3} = 0.67$$

请计算置信度

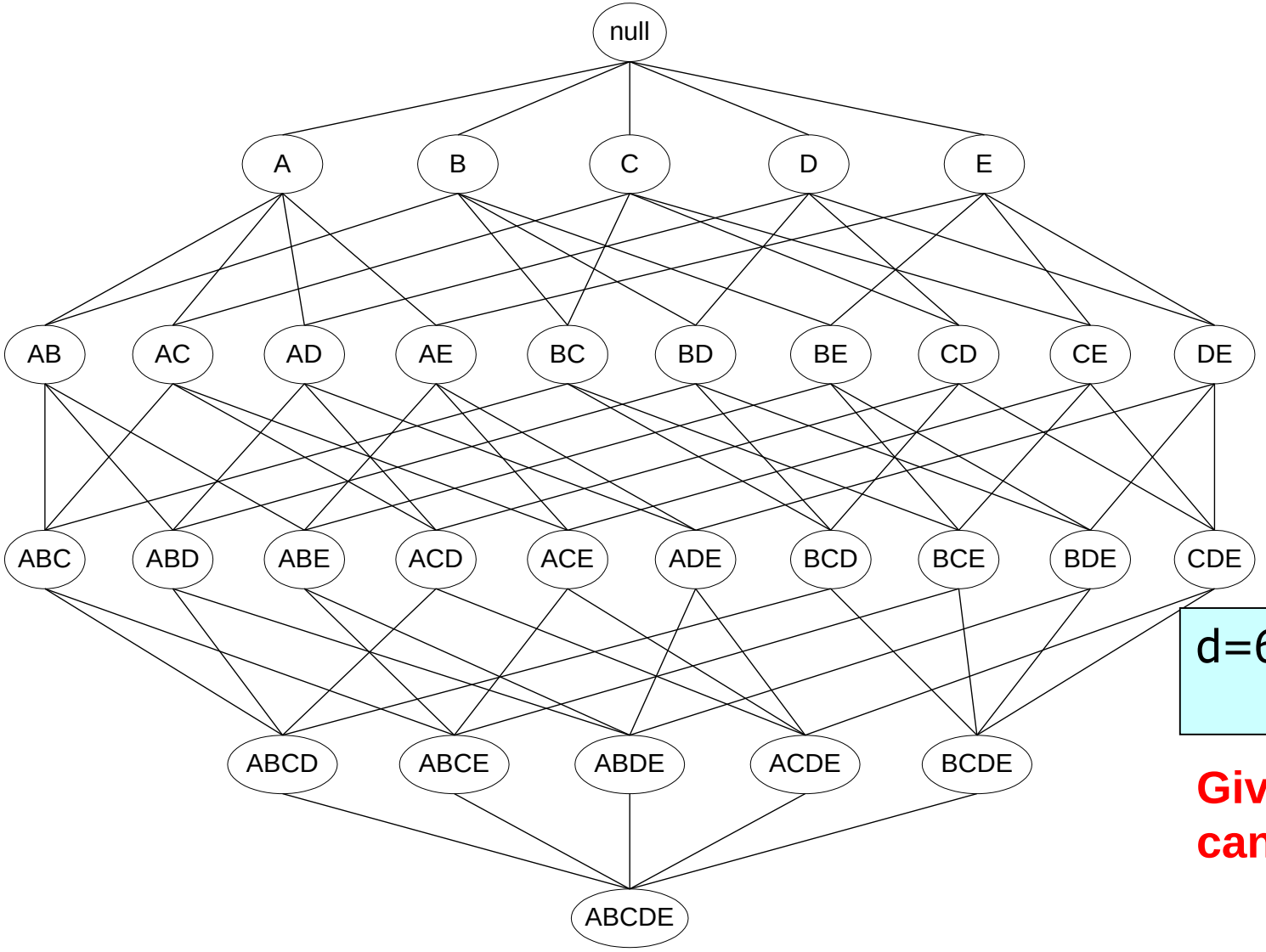
支持度表示所分析的所有事务数据的s显示X和Y一起
置信度表示如果X，则Y的可能性

# Association Rule Mining Task

- Given a set of transactions T, the goal of association rule mining is to find all rules having
  - —support ≥ *minsup* threshold
  - —confidence ≥ *minconf* threshold

- Brute-force（蛮力、暴力） approach:
  - —List all possible association rules
  - —Compute the support and confidence for each rule
  - —Prune rules that fail the *minsup* and *minconf* thresholds

# Frequent Itemset Generation
# 格结构（lattice structure）常常被用来枚举所有可能的项集




{Milk, Bread, Diaper}

{Milk,Diaper} → {Beer} (s=0.4, c=0.67)
{Milk,Beer} → {Diaper} (s=0.4, c=1.0)
{Diaper,Beer} → {Milk} (s=0.4, c=0.67)
{Beer} → {Milk,Diaper} (s=0.4, c=0.67)
{Diaper} → {Milk,Beer} (s=0.4, c=0.5)
{Milk} → {Diaper,Beer} (s=0.4, c=0.5)

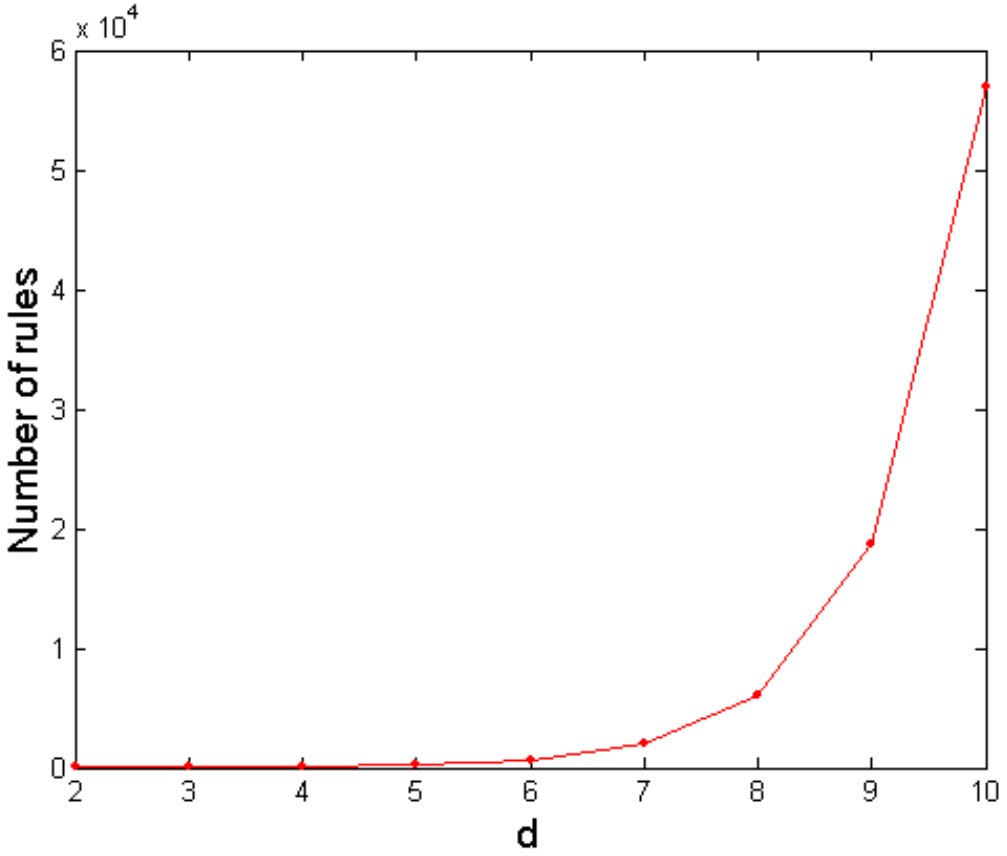
d=6，If every subset is considered,

$${}^6C_1 + {}^6C_2 + {}^6C_3 + {}^6C_4 + {}^6C_5 + {}^6C_6$$

**Given d items, there are $2^d$ possible candidate itemsets**

# Association Rules Generation Computational Complexity

- **Given d unique items:**
  - **Total number of itemsets = $2^d$**
  - **Total number of possible association rules:**



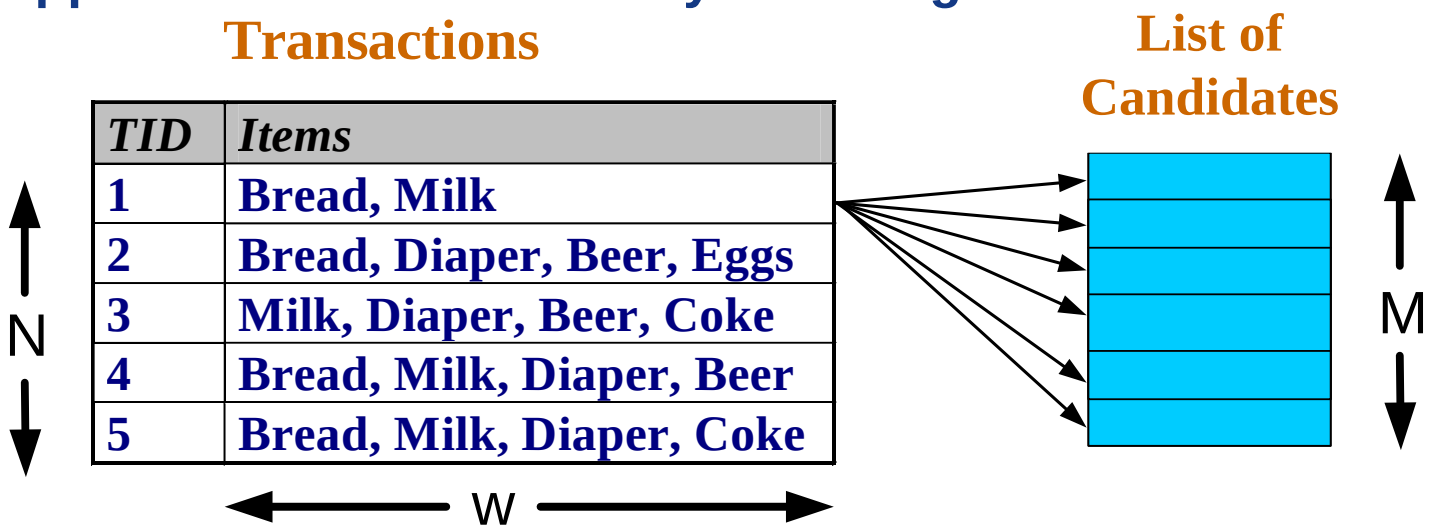

$$R = \sum_{k=1}^{d-1}\left[\binom{d}{k} \times \sum_{j=1}^{d-k}\binom{d-k}{j}\right]$$

$$= 3^d - 2^{d+1} + 1$$

**If d=6, R = 602 rules**

# Frequent Itemset Generation Computational Complexity

- **Brute-force approach:**
  - Each itemset in the lattice is a **candidate** frequent itemset
  - Count the support of each candidate by scanning the database



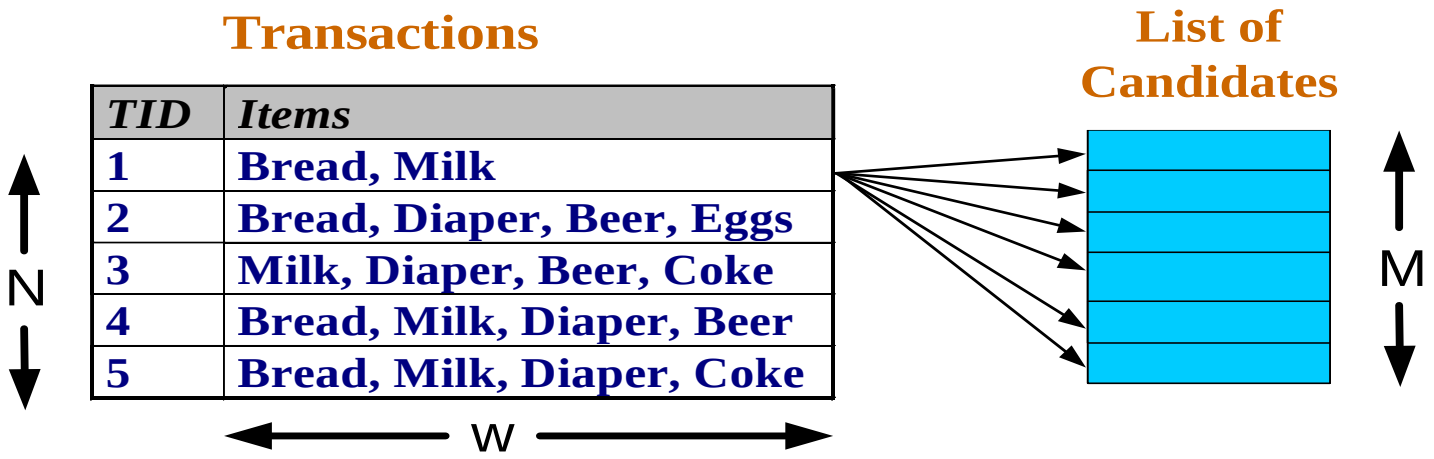

  - Match each transaction against every candidate
  - Complexity ~ O(NMw) => **Expensive since $M = 2^d$ !!!**

- ⇒ **Computationally prohibitive（禁止的、过高的）!**

# Frequent Itemset Generation Strategies

## Complexity ~ O(NMw)

- Reduce the number of candidates (M)
  - —Complete search: $M=2^d$
  - —Use pruning techniques to reduce M
- Reduce the number of comparisons (NW)
  - —Use efficient data structures to store the candidates or transactions
  - —No need to match every candidate against every transaction



# Improvement methods 1

# Mining Association Rules

| TID | Items |
|---|---|
| 1 | Bread, Milk |
| 2 | Bread, Diaper, Beer, Eggs |
| 3 | Milk, Diaper, Beer, Coke |
| 4 | Bread, Milk, Diaper, Beer |
| 5 | Bread, Milk, Diaper, Coke |

## Example of Rules:

{Milk,Diaper} → {Beer} (s=0.4, c=0.67)
{Milk,Beer} → {Diaper} (s=0.4, c=1.0)
{Diaper,Beer} → {Milk} (s=0.4, c=0.67)
{Beer} → {Milk,Diaper} (s=0.4, c=0.67)
{Diaper} → {Milk,Beer} (s=0.4, c=0.5)
{Milk} → {Diaper,Beer} (s=0.4, c=0.5)

## Observations:

- All the above rules are binary（双重、二元） partitions of the same itemset:
  {Milk, Diaper, Beer}
- Rules originating from the same itemset have identical support but can have different confidence
- Thus, we may decouple（解耦） the support and confidence requirements

# Mining Association Rules

- **Two-step approach:**
  1. **Frequent Itemset Generation**
     - **Generate all itemsets whose support ≥ minsup**
  2. **Rule Generation**
     - **Generate high confidence rules from each frequent itemset, where each rule is a binary partitioning of a frequent itemset**

- **Frequent itemset generation is still computationally expensive（计算量太大）**

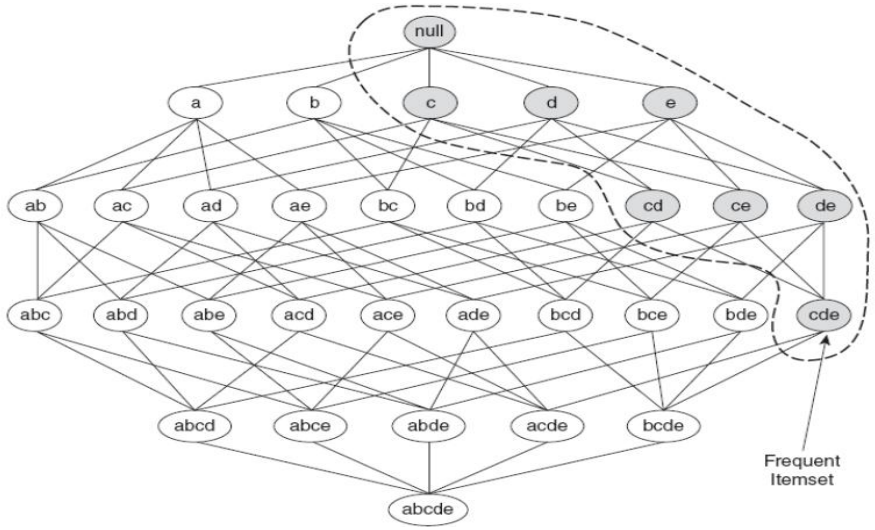
# Improvement methods 2

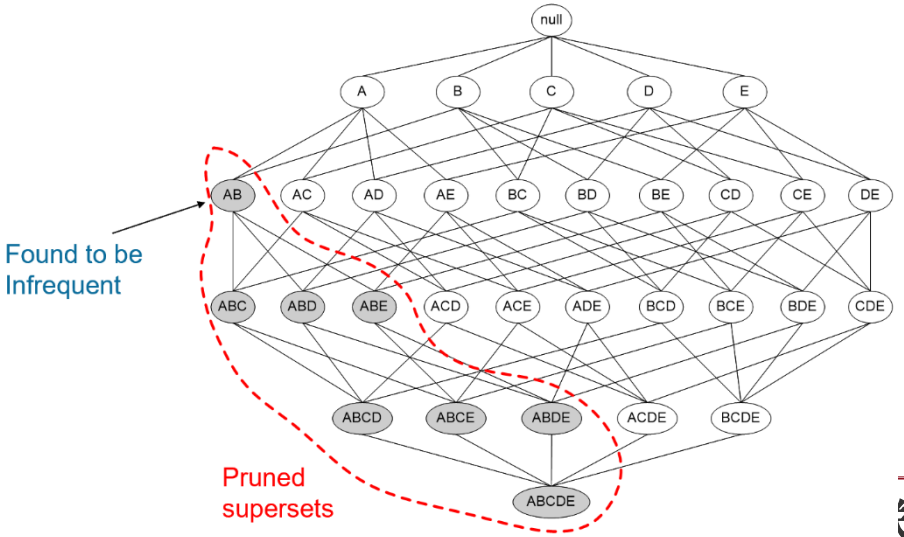
# （1）Reducing Number of Candidates

- Apriori 美['eɪprɪ'ɔ:rɪ] adj.<拉>由原因推及结果的;演绎的;先验的;推测的
- **Apriori principle:**
  - **If an itemset is frequent, then all of its subsets must also be frequent**
- **Apriori principle holds due to the following property of the support measure:**

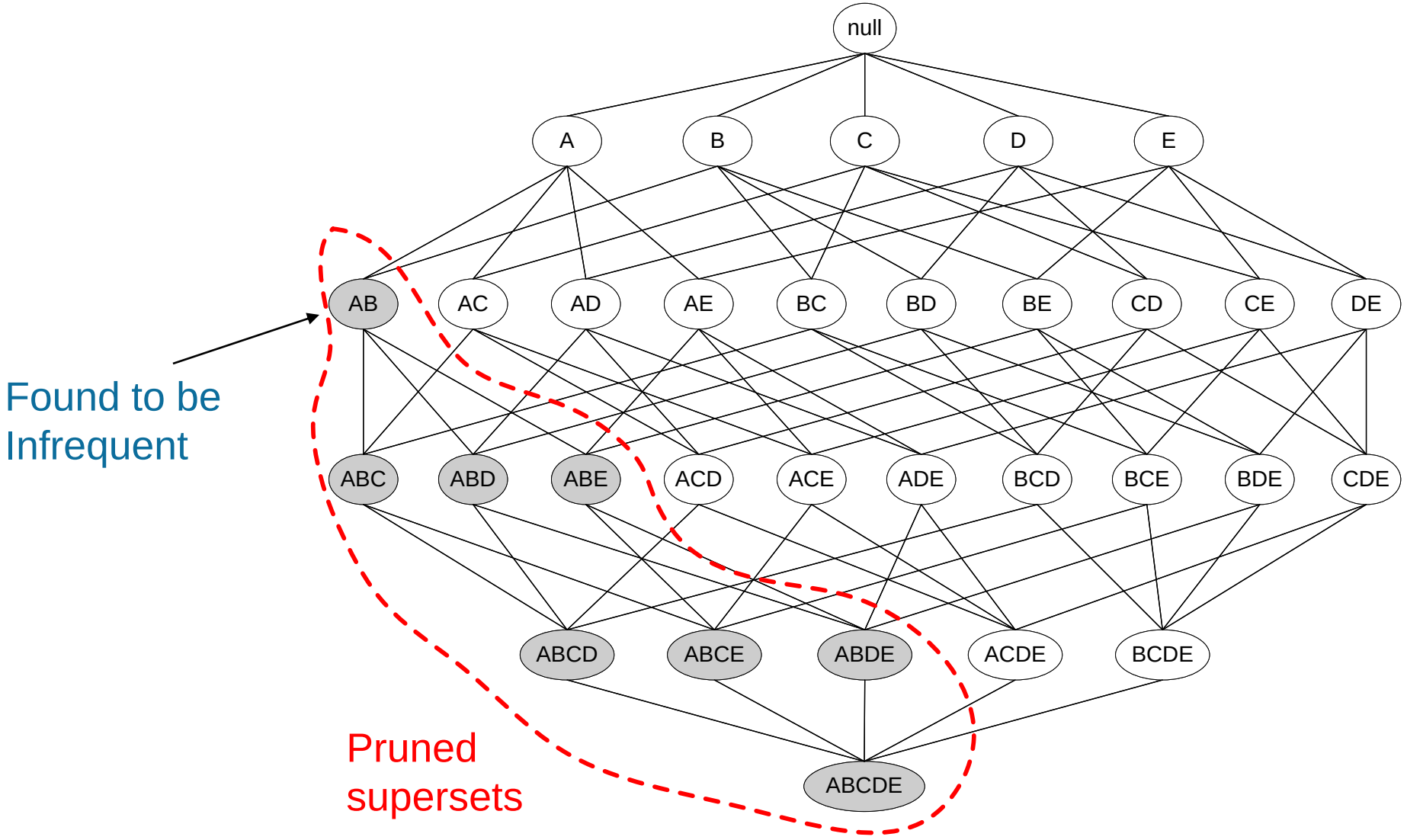
$$\forall X, Y : (X \subseteq Y) \Rightarrow s(X) \geq s(Y)$$

  - **Support of an itemset never exceeds the support of its subset**
  - **Itemset which is infrequent, its superset is also infrequent**
  - 一个集合不能通过测试，它的所有超集也都不能通过相同测试（反单调）
  - **This is known as the anti-monotone property of support**



# Illustrating Apriori Principle



# Illustrating Apriori Principle

| Item | Count |
|---|---|
| Bread | 4 |
| Coke | 2 |
| Milk | 4 |
| Beer | 3 |
| Diaper | 4 |
| Eggs | 1 |

Items (1-itemsets)

Minimum Support = 3

| Itemset | Count |
|---|---|
| {Bread,Milk} | 3 |
| {Bread,Beer} | 2 |
| {Bread,Diaper} | 3 |
| {Milk,Beer} | 2 |
| {Milk,Diaper} | 3 |
| {Beer,Diaper} | 3 |

Pairs (2-itemsets)

(No need to generate candidates involving Coke or Eggs)

| Itemset | Count |
|---|---|
| {Bread,Milk,Diaper} | 2 |

Triplets (3-itemsets)

(No need to generate candidates involving {Bread,Beer} or {Milk,Beer} )

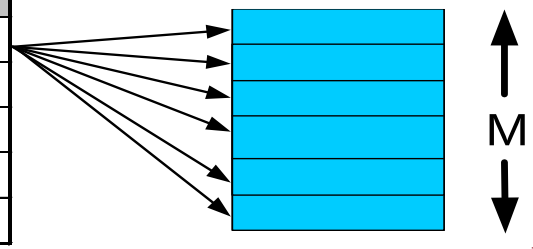
**Transactions**

| *TID* | *Items* |
|---|---|
| 1 | Bread, Milk |
| 2 | Bread, Diaper, Beer, Eggs |
| 3 | Milk, Diaper, Beer, Coke |
| 4 | Bread, Milk, Diaper, Beer |
| 5 | Bread, Milk, Diaper, Coke |

N

W

**List of Candidates**

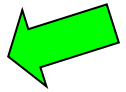


M

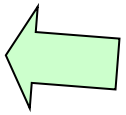
# Apriori Algorithm

- **Method:**
  - Let k=1
  - Generate frequent itemsets of length 1
  - Repeat until no new frequent itemsets are identified
    - Generate length (k+1) candidate itemsets from length k frequent itemsets
    - Prune candidate itemsets containing subsets of length k that are infrequent
    - Count the support of each candidate by scanning the DB
    - Eliminate candidates that are infrequent, leaving only those that are frequent

# Chapter 6
# 挖掘频繁模式、关联和相关性基本概念和方法

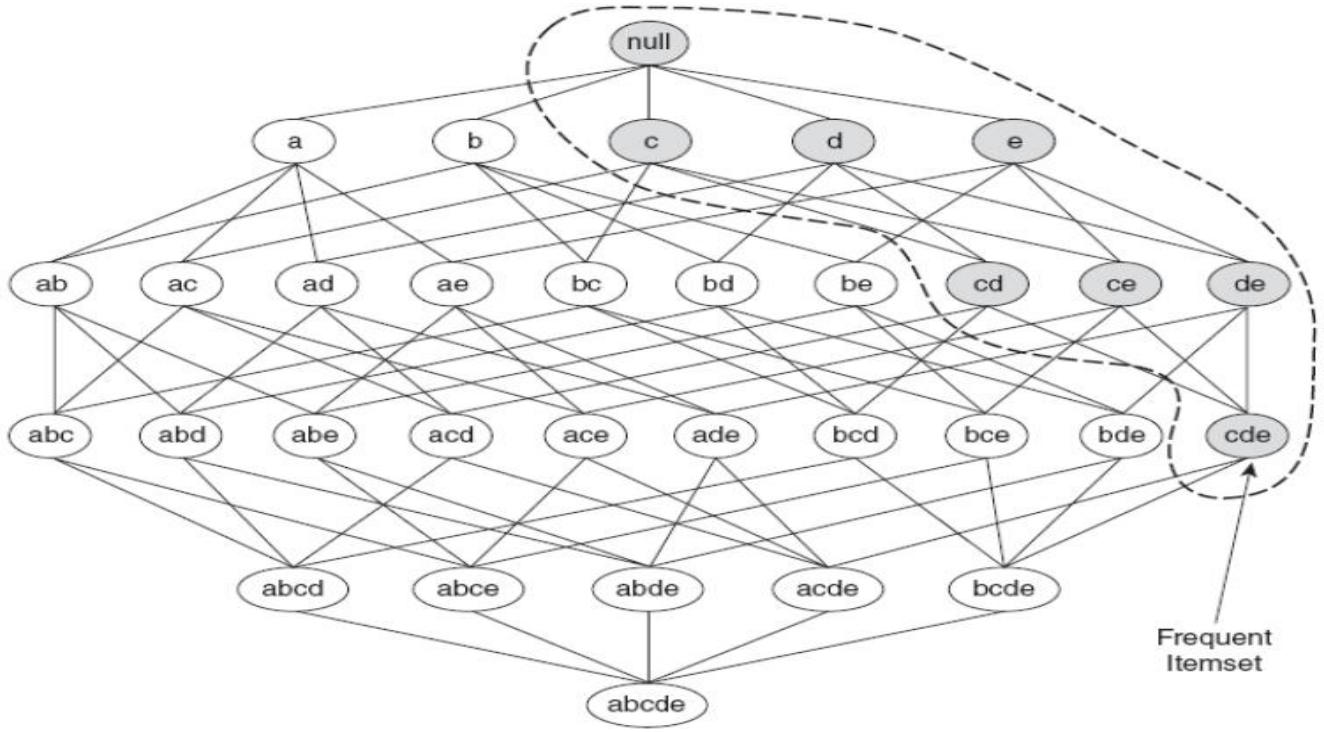
- 基本概念
- 频繁项集挖掘方法 
- 哪些模式是有趣的：模式评估方法
- 小结

# 可扩展的频繁项集挖掘方法

- Apriori:候选生成和测试方法 
- 提升Apriori的效率
- FPGrowth:频繁模式增长的方法
- ECLAT:垂直数据格式下的频繁模式挖掘

# 向下封闭性和可扩展的挖掘方法

- 频繁模式的先验性质（向下封闭性）
  - —频繁项集的所有非空子集也一定是频繁的
  - —If {beer, diaper, nuts} is frequent, so is {beer, diaper}
  - —i.e., every transaction having {beer, diaper, nuts} also contains {beer, diaper}
- 可扩展的挖掘方法: 三个主要方法
  - —**Apriori (先验算法)**
  - —**Freq. pattern growth**
  - —**Vertical data format approach**



# Apriori: 通过限制候选产生发现频繁项集

- Apriori 美['eɪprɪ'ɔ:rɪ] adj.<拉>由原因推及结果的;演绎的;先验的;推测的
- **Apriori剪枝原则: If there is any itemset which is infrequent, its superset should not be generated/tested**
- 方法:
  - 最初，一次扫描数据库得到频繁1 - 项集
  - 从长度为k的频繁项集中生成长度（k +1）的候选项集
  - 根据数据测试候选项集
  - 当没有频繁或候选集生成时，算法结束

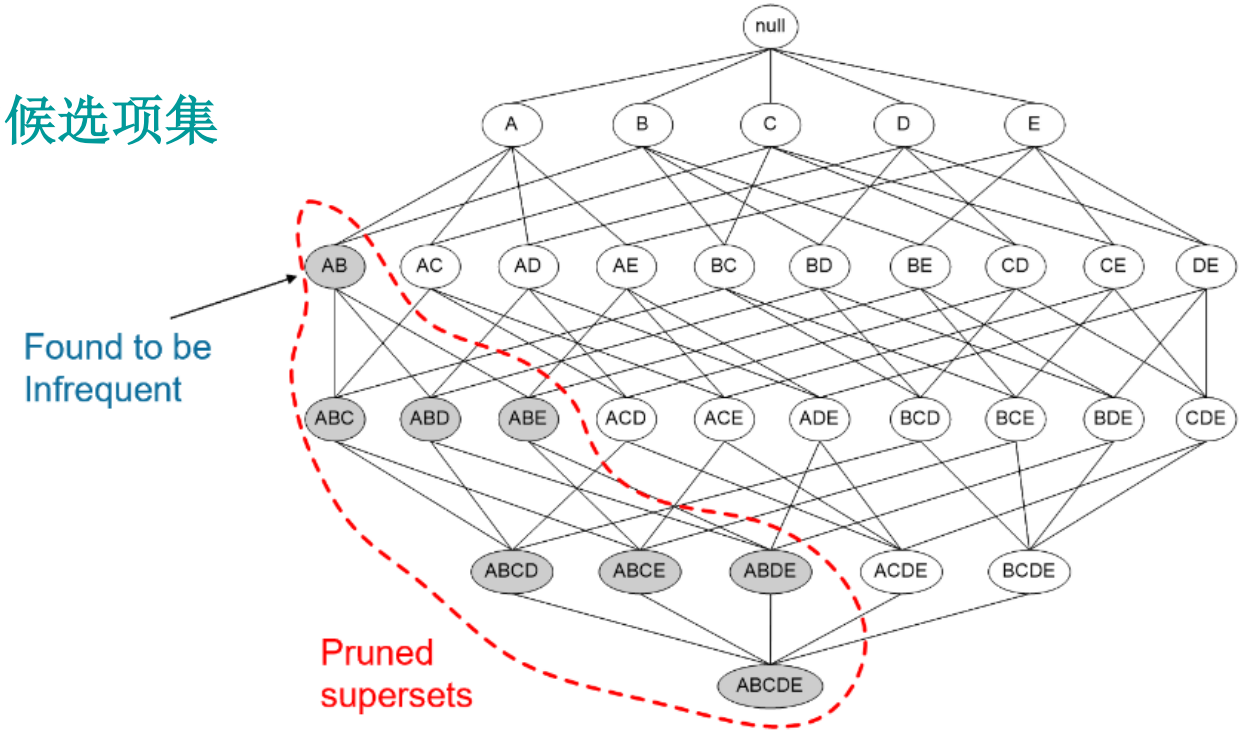


表 6.1　AllElectronics 某分店的事务数据

| TID | 商品 ID 的列表 | TID | 商品 ID 的列表 |
|---|---|---|---|
| T100 | I1，I2，I5 | T600 | I2，I3 |
| T200 | I2，I4 | T700 | I1，I3 |
| T300 | I2，I3 | T800 | I1，I2，I3，I5 |
| T400 | I1，I2，I4 | T900 | I1，I2，I3 |
| T500 | I1，I3 | | |

**Ck**: 长度为k的候选集
**Lk**：长度为k的频繁项集

$C_1$

| 项集 | 支持度计数 |
|---|---|
| {I1} | 6 |
| {I2} | 7 |
| {I3} | 6 |
| {I4} | 2 |
| {I5} | 2 |

扫描D，对每个候选计数 →

候选的支持度计数与最小支持度计数比较 →

$L_1$

| 项集 | 支持度计数 |
|---|---|
| {I1} | 6 |
| {I2} | 7 |
| {I3} | 6 |
| {I4} | 2 |
| {I5} | 2 |

$Sup_{min} = 2$

由$L_1$产生候选$C_2$ →

$C_2$

| 项集 |
|---|
| {I1, I2} |
| {I1, I3} |
| {I1,I4} |
| {I1, I5} |
| {I2, I3} |
| {I2, I4} |
| {I2, I5} |
| {I3, I4} |
| {I3, I5} |
| {I4, I5} |

扫描D，对每个候选计数 →

$C_2$

| 项集 | 支持度计数 |
|---|---|
| {I1, I2} | 4 |
| {I1, I3} | 4 |
| {I1, I4} | 1 |
| {I1, I5} | 2 |
| {I2, I3} | 4 |
| {I2, I4} | 2 |
| {I2, I5} | 2 |
| {I3, I4} | 0 |
| {I3, I5} | 1 |
| {I4, I5} | 0 |

候选的支持度计数与最小支持度计数比较 →

$L_2$

| 项集 | 支持度计数 |
|---|---|
| {I1, I2} | 4 |
| {I1, I3} | 4 |
| {I1, I5} | 2 |
| {I2, I3} | 4 |
| {I2, I4} | 2 |
| {I2, I5} | 2 |

(a) 连接：$C_3=L_2 \bowtie L_2=\{\{I1, I2\}, \{I1, I3\}, \{I1, I5\}, \{I2, I3\}, \{I2, I4\}, \{I2, I5\}\}$

$\bowtie\{\{I1, I2\}, \{I1, I3\}, \{I1, I5\}, \{I2, I3\}, \{I2, I4\}, \{I2, I5\}\}$

{I1,I2,I4} $=\{\{I1, I2, I3\}, \{I1, I2, I5\}, \{I1, I3, I5\}, \{I2, I3, I4\}, \{I2, I3, I5\}, \{I2, I4, I5\}\}$

(b) 使用先验性质剪枝：频繁项集的所有非空子集必须是频繁的。存在候选项集，其子集不是频繁的吗?

- $\{I1, I2, I3\}$的2项子集是$\{I1, I2\}$、$\{I1, I3\}$和$\{I2, I3\}$。$\{I1, I2, I3\}$的所有2项子集都是$L_2$的元素。因此，$\{I1, I2, I3\}$保留在$C_3$中。
- $\{I1, I2, I5\}$的2项子集是$\{I1, I2\}$、$\{I1, I5\}$和$\{I2, I5\}$。$\{I1, I2, I5\}$的所有2项子集都是$L_2$的元素。因此，$\{I1, I2, I5\}$保留在$C_3$中。
- $\{I1, I3, I5\}$的2项子集是$\{I1, I3\}$、$\{I1, I5\}$和$\{I3, I5\}$。$\{I3, I5\}$不是$L_2$的元素，因而不是频繁的。因此，从$C_3$中删除$\{I1, I3, I5\}$。
- $\{I2, I3, I4\}$的2项子集是$\{I2, I3\}$、$\{I2, I4\}$和$\{I3, I4\}$。$\{I3, I4\}$不是$L_2$的元素，因而不是频繁的。因此，从$C_3$中删除$\{I2, I3, I4\}$。
- $\{I2, I3, I5\}$的2项子集是$\{I2, I3\}$、$\{I2, I5\}$和$\{I3, I5\}$。$\{I3, I5\}$不是$L_2$的元素，因而不是频繁的。因此，从$C_3$中删除$\{I2, I3, I5\}$。
- $\{I2, I4, I5\}$的2项子集是$\{I2, I4\}$、$\{I2, I5\}$和$\{I4, I5\}$。$\{I4, I5\}$不是$L_2$的元素，因而不是频繁的。因此，从$C_3$中删除$\{I2, I3, I5\}$。

(c) 因此，剪枝后$C_3=\{\{I1, I2, I3\}, \{I1, I2, I5\}\}$。

由$L_2$产生候选$C_3$ →

$C_3$

| 项集 |
|---|
| {I1, I2, I3} |
| {I1, I2, I5} |

扫描$D$，对每个候选计数 →

$C_3$

| 项集 | 支持度计数 |
|---|---|
| {I1, I2, I3} | 2 |
| {I1, I2, I5} | 2 |

候选的支持度计数与最小支持度计数比较 →

$L_3$

| 项集 | 支持度计数 |
|---|---|
| {I1, I2, I3} | 2 |
| {I1, I2, I5} | 2 |

# Apriori算法(伪代码)

$C_k$: 长度为k的候选集
$L_k$: 长度为k的频繁项集

$L_1$ = {频繁项};
for ($k$ = 1; $L_k$ !=∅; $k$++) do begin
    $C_{k+1}$ =从$L_k$生成候选集;
    for each 数据库中的事务 $t$ do
        增加所有$C_{k+1}$ 中t包含的候选集的count
    $L_{k+1}$ = $C_{k+1}$ 中大于min_support的候选集
    end
return $\cup_k L_k$;

# Apriori实现

- 如何生成候选项集?
  - —Step 1: 连接步 $L_k$
  - —Step 2: 剪枝步
- Example of Candidate-generation
  - —$L_3$={*abc, abd, acd, ace, bcd*}
  - —Self-joining: $L_3$*$L_3$
    - *abcd* from *abc* and *abd*
    - *acde* from *acd* and *ace*
  - —Pruning:
    - *acde* is removed because *ade* is not in $L_3$
  - —$C_4$ = {*abcd*}

abcd,abce
0
acde
0

# 候选集生成: An SQL Implementation

- 候选集生成的SQL 实现
  - Suppose the items in $L_{k-1}$ are listed in an order
  - Step 1: self-joining $L_{k-1}$

    insert into $C_k$

    select $p.item_1, p.item_2, \ldots, p.item_{k-1}, q.item_{k-1}$

    from $L_{k-1}$ p, $L_{k-1}$ q

    where $p.item_1=q.item_1, \ldots, p.item_{k-2}=q.item_{k-2}, p.item_{k-1} < q.item_{k-1}$
  - Step 2: pruning

    forall *itemsets c in* $C_k$ do

    forall ***(k-1)-subsets s of c*** do

    **if** (*s is not in* $L_{k-1}$) **then delete** *c* **from** $C_k$
- **Use object-relational extensions like UDFs, BLOBs, and Table functions for efficient implementation [See: S. Sarawagi, S. Thomas, and R. Agrawal. Integrating association rule mining with relational database systems: Alternatives and implications. SIGMOD'98]**

# The Apriori Algorithm—随堂练习

Database TDB

| Tid | Items |
|---|---|
| 10 | A, C, D |
| 20 | B, C, E |
| 30 | A, B, C, E |
| 40 | B, E |

$Sup_{min} = 2$

# 由频繁项集产生关联规则

- 频繁项集X={I1,I2,I5}的非空子集
  - —{I1,I2}
  - —{I1,I5}
  - —{I2,I5}
  - —{I1}
  - —{I2}
  - —{I5}

$\{I1, I2\} \Rightarrow I5$, confidence = 2/4 = 50%

$\{I1, I5\} \Rightarrow I2$, confidence = 2/2 = 100%

$\{I2, I5\} \Rightarrow I1$, confidence = 2/2 = 100%

$I1 \Rightarrow \{I2, I5\}$, confidence = 2/6 = 33%

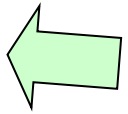
$I2 \Rightarrow \{I1, I5\}$, confidence = 2/7 = 29%

$I5 \Rightarrow \{I1, I2\}$, confidence = 2/2 = 100%

表 6.1 AllElectronics 某分店的事务数据

| *TID* | 商品 *ID* 的列表 | *TID* | 商品 *ID* 的列表 |
|---|---|---|---|
| T100 | I1, I2, I5 | T600 | I2, I3 |
| T200 | I2, I4 | T700 | I1, I3 |
| T300 | I2, I3 | T800 | I1, I2, I3, I5 |
| T400 | I1, I2, I4 | T900 | I1, I2, I3 |
| T500 | I1, I3 | | |

# 可扩展的频繁项集挖掘方法

- Apriori:候选生成和测试方法
- 提升Apriori的效率 
- FPGrowth:频繁模式增长的方法
- ECLAT:垂直数据格式下的频繁模式挖掘

# 提高Apriori算法的有效性

- Apriori算法主要的挑战
  - —要对数据进行多次扫描；
  - —会产生大量的候选项集；
  - —对候选项集的支持度计算非常繁琐；
- 解决思路
  - —减少对数据的扫描次数；
  - —缩小产生的候选项集；
  - —改进对候选项集的支持度计算方法

# 提高Apriori算法的有效性(1)

- **方法1：基于hash表的项集计数**
  - —将每个项集通过相应的hash函数映射到hash表中的不同的桶中，这样可以通过将桶中的项集计数跟最小支持计数相比较先淘汰一部分项集。

# 散列项集到对应的桶中

- 当扫描数据库中每个事务由C1中的候选1项集产生频繁1项集时，可以对每个事务产生所有的2项集，将它们散列（即映射）到散列表结构的不同桶中，并增加对应的桶计数
- 在散列表中，对应的桶计数低于支持度阈值的2项集不可能是频繁的，因此应该从候选集中删除
- 这种基于散列的技术可以显著地压缩需要考察的K项集（特别是K=2时）

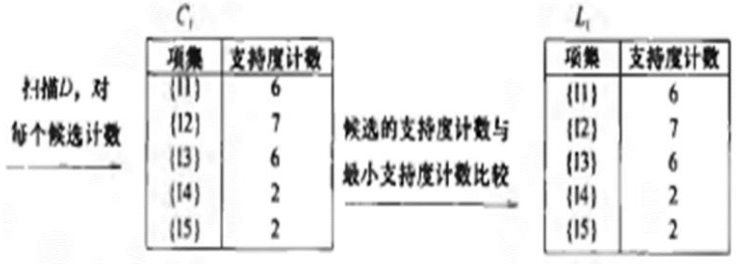



表 6.1 AllElectronics 某分店的事务数据

| TID | 商品 ID 的列表 | TID | 商品 ID 的列表 |
|---|---|---|---|
| T100 | I1, I2, I5 | T600 | I2, I3 |
| T200 | I2, I4 | T700 | I1, I3 |
| T300 | I2, I3 | T800 | I1, I2, I3, I5 |
| T400 | I1, I2, I4 | T900 | I1, I2, I3 |
| T500 | I1, I3 | | |

使用散列函数

hash(x, y)=((order of x) *10+(order of y)) mod 7

创建散列表 $H_2$

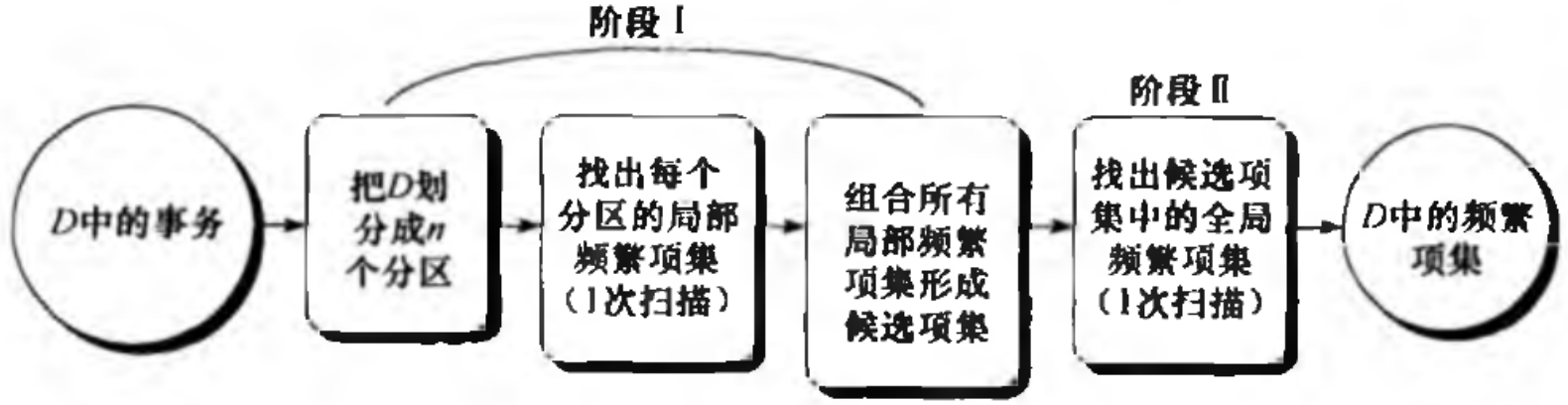
$H_2$

| 桶地址 | 0 | 1 | 2 | 3 | 4 | 5 | 6 |
|---|---|---|---|---|---|---|---|
| 桶计数 | 2 | 2 | 4 | 2 | 2 | 4 | 4 |
| 桶内容 | {I1,I4}<br>{I3,I5} | {I1,I5}<br>{I1,I5} | {I2,I3}<br>{I2,I3}<br>{I2,I3}<br>{I2,I3} | {I2,I4}<br>{I2,I4} | {I2,I5}<br>{I2,I5} | {I1,I2}<br>{I1,I2}<br>{I1,I2}<br>{I1,I2} | {I1,I3}<br>{I1,I3}<br>{I1,I3}<br>{I1,I3} |

如果最小支持度为3，则桶0 、1、3 和4 中的项集不可能是频繁的，因此它们不包含在C2中

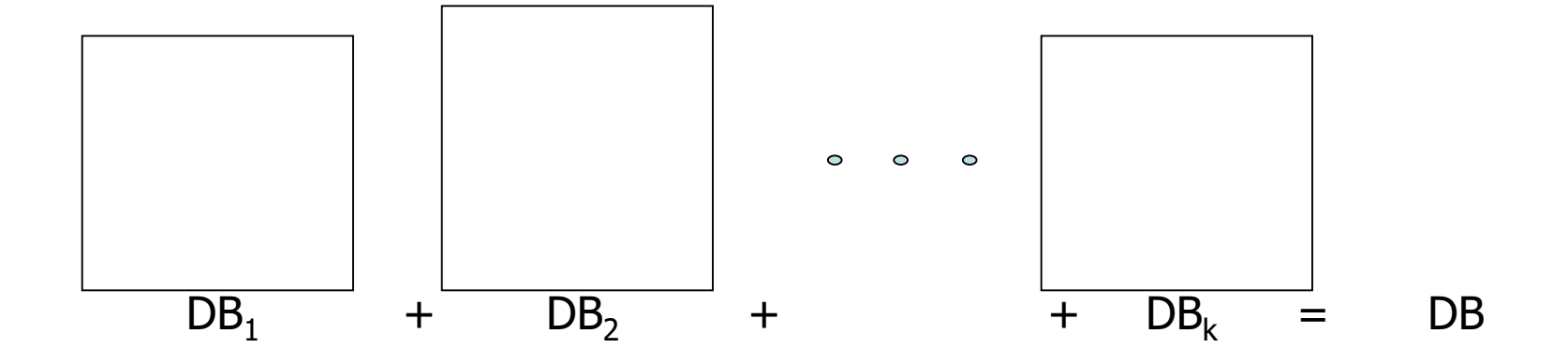
# 提高Apriori算法的有效性(2)

- 方法2：划分
  - —D中的任何频繁项集必须作为局部频繁项集至少出现在一个部分中
    - 第一次扫描：将数据划分为多个部分并找到局部频繁项集
    - 第二次扫描：评估每个候选项集的实际支持度，以确定全局频繁项集
  - —挖掘频繁项集只需要两次数据扫描



# 划分: 只扫描两次数据库

- 数据库中可能频繁的项集，至少在一个数据集的划分中也是频繁的
  - —**Scan 1:** 将数据集划分，找到局部频繁项集
  - —**Scan 2:** 在全局中巩固计算结果
- **A. Savasere, E. Omiecinski and S. Navathe, *VLDB'95***



# 提高Apriori算法的有效性(3)

- 方法3：选样（在给定数据的一个子集挖掘）
  - —基本思想：选择原始数据的一个样本，在这个样本上用Apriori算法挖掘频繁模式
  - —通过牺牲精确度来减少算法开销，为了提高效率，样本大小应该以可以放在内存中为宜，可以适当降低最小支持度来减少遗漏的频繁模式
    - 可以通过一次全局扫描来验证从样本中发现的模式
    - 可以通过第二此全局扫描来找到遗漏的模式

# 抽样

- 对数据库进行抽样, 使用Apriori方法对样本进行频繁模式挖掘
- 扫描数据库，验证样本中挖掘出的频繁模式结果，检查闭频繁模式Example: check *abcd* instead of *ab*, *ac*, …, etc.（*频繁模式的先验性质—向下封闭性：频繁项集的所有非空子集也一定是频繁的*）
- 再次扫描数据库，找到遗漏的频繁模式
- **H. Toivonen. Sampling large databases for association rules. In *VLDB'96***

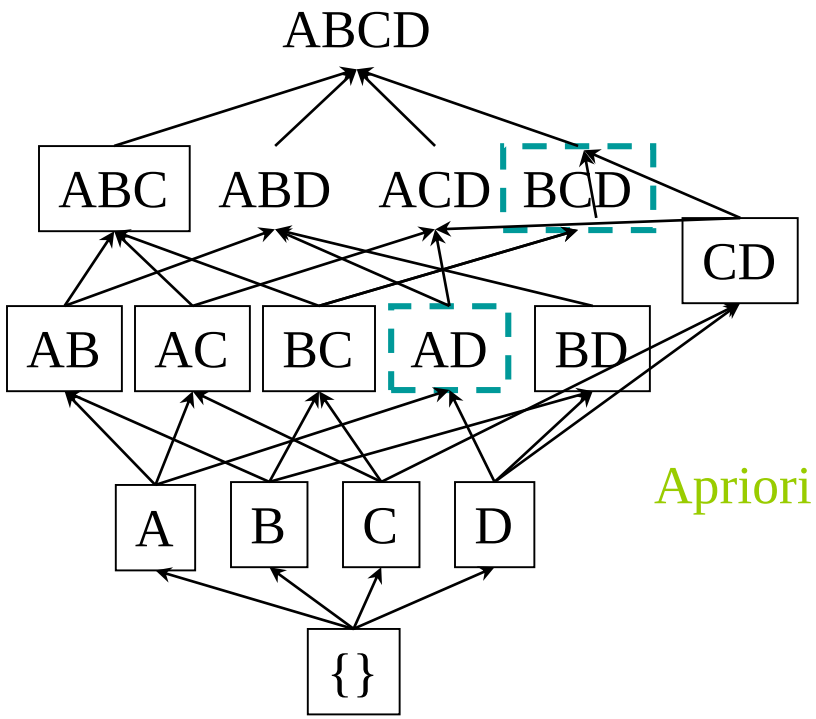
# 提高Apriori算法的有效性(4)

- 方法4：事务压缩（压缩进一步迭代的事务数）
  - —不包含任何频繁k-项集的事务不可能包含任何频繁(k+1)-项集，这种事务在下一步的计算中可以加上标记或删除

# 提高Apriori算法的有效性(5)

- 方法5：动态项集计数(在扫描的不同点添加候选项集)
  - —动态项集计数技术将数据库划分为用开始点标记的块。不像Apriori算法仅在每次完整的数据库扫描前确定新的候选；在这种变形中，可以在任何开始点添加新的候选项集。该技术使用迄今为止的计数作为实际计数的下界。如果迄今为止的计数满足最小支持度，则该项集添加到频繁项集的集合中，并且可以用来产生更长的候选
  - —为了找出所有的频繁项集，需要的数据库扫描比Apriori算法少

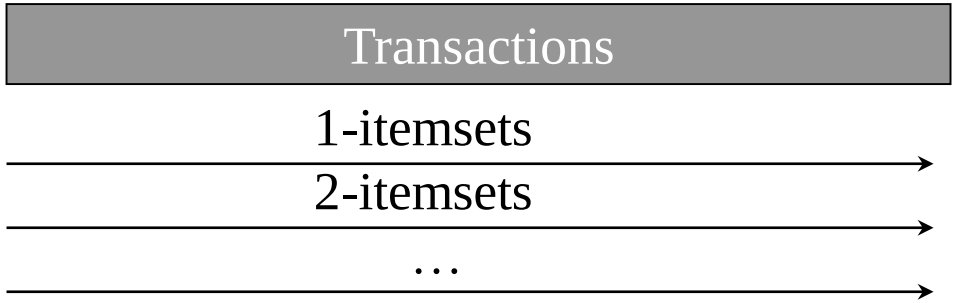
# 减少扫描次数



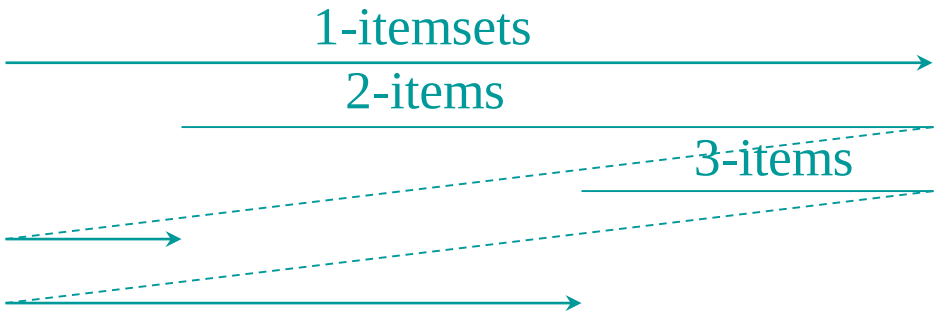

Itemset lattice

- 一旦A和D确定是频繁的，即开始计算AD的数量
- 一旦BCD包含的长度为2的所有子集都确定是频繁的，即开始进行BCD的计数

Apriori

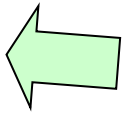



DIC



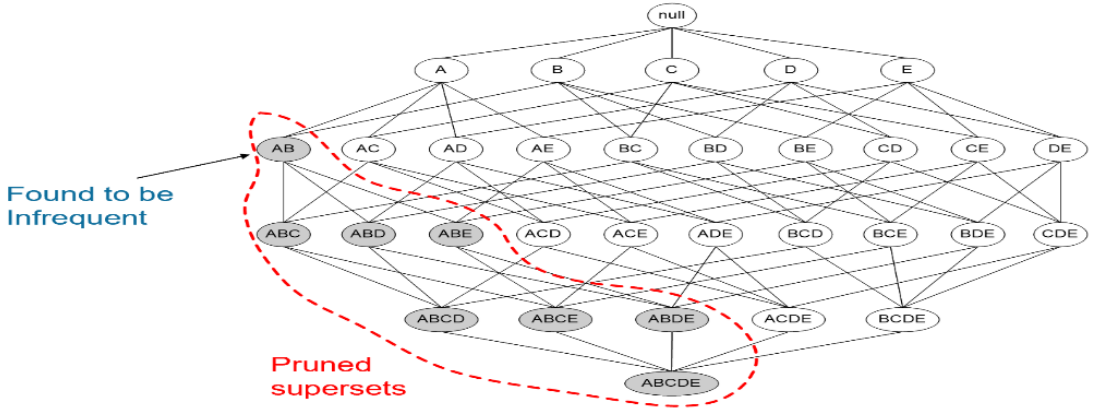

S. Brin R. Motwani, J. Ullman, and S. Tsur. Dynamic itemset counting and implication rules for market basket data. *SIGMOD'97*

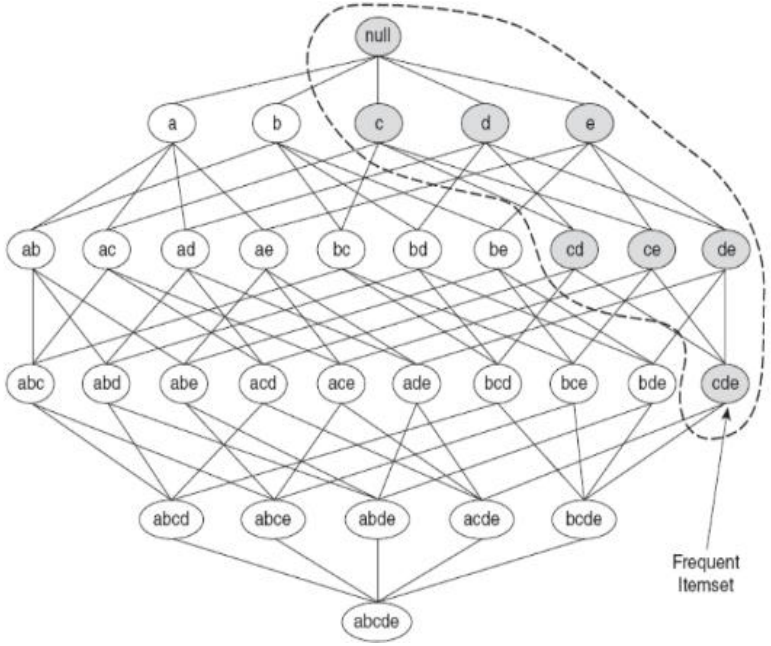
# 可扩展的频繁项集挖掘方法

- Apriori:候选生成和测试方法
- 提升Apriori的效率
- FPGrowth:频繁模式增长的方法 
- ECLAT:垂直数据格式下的频繁模式挖掘

# 模式增长方法:
# 不使用生成候选集挖掘频繁项集

- Apriori方法的瓶颈
  - 广度优先(i.e., level-wise) 搜索
  - 使用候选集生成
    - 可能会产生大量候选集
- FPGrowth方法(J. Han, J. Pei, and Y. Yin)
  - Frequent-Pattern Growth，频繁模式增长
  - 深度优先搜索
  - 避免显式地产生候选
- 主要理念: 从短的模式中使用局部频繁项，得到长的模式
  - “abc” is a frequent pattern
  - Get all transactions having “abc”, i.e., project DB on abc: DB|abc
  - “d” is a local frequent item in DB|abc → abcd is a frequent pattern



# FP-growth Algorithm

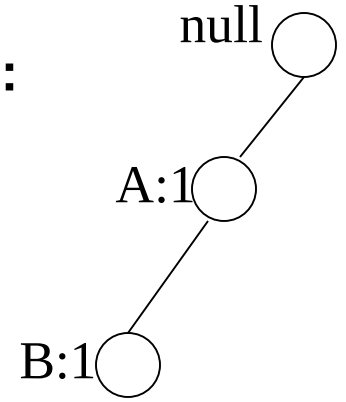
- Use a compressed representation of the database using an FP-tree

- Once an FP-tree has been constructed, it uses a recursive（递归） divide-and-conquer（分治） approach to mine the frequent itemsets

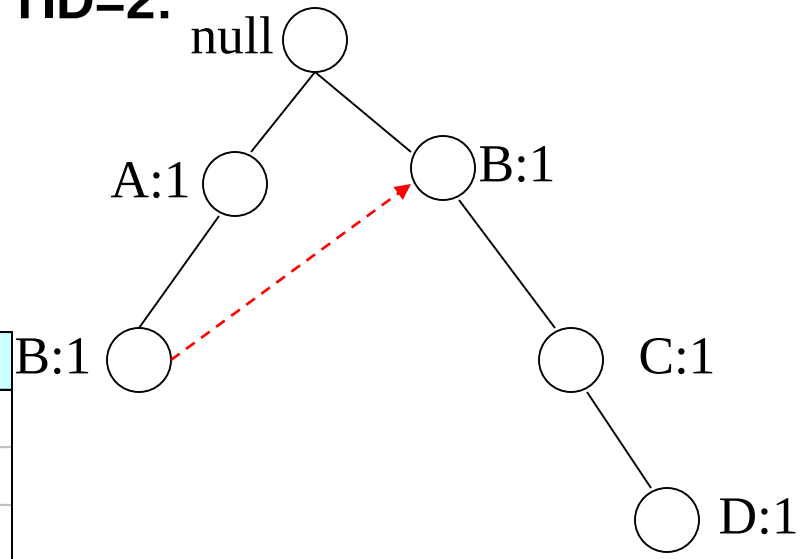
# FP-tree construction

| TID | Items |
|---|---|
| 1 | {A,B} |
| 2 | {B,C,D} |
| 3 | {A,C,D,E} |
| 4 | {A,D,E} |
| 5 | {A,B,C} |
| 6 | {A,B,C,D} |
| 7 | {A} |
| 8 | {A,B,C} |
| 9 | {A,B,D} |
| 10 | {B,C,E} |

**After reading TID=1:**




**After reading TID=2:**




| Item | Count | Pointer |
|---|---|---|
| A | 8 | |
| B | 7 | |
| C | 6 | |
| D | 5 | |
| E | 3 | |

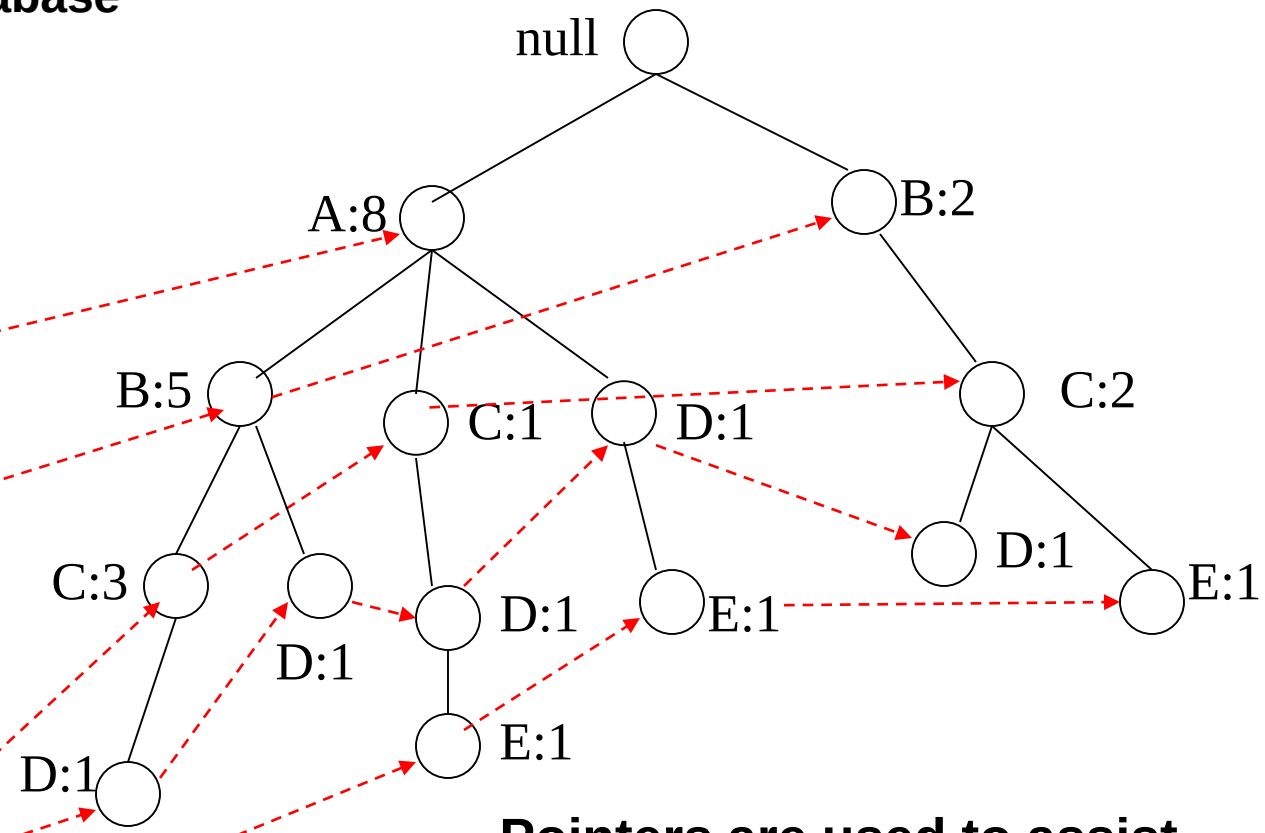
# FP-Tree Construction

| TID | Items |
|---|---|
| 1 | {A,B} |
| 2 | {B,C,D} |
| 3 | {A,C,D,E} |
| 4 | {A,D,E} |
| 5 | {A,B,C} |
| 6 | {A,B,C,D} |
| 7 | {A} |
| 8 | {A,B,C} |
| 9 | {A,B,D} |
| 10 | {B,C,E} |

**Transaction Database**

**Header table（项头表）**

| Item | Count | Pointer |
|---|---|---|
| A | 8 | |
| B | 7 | |
| C | 6 | |
| D | 5 | |
| E | 3 | |




**Pointers are used to assist frequent itemset generation**

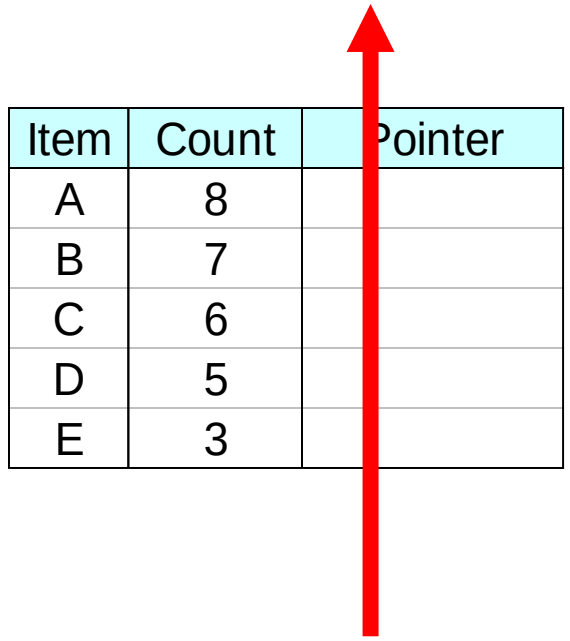
# FP-Tree Construction

- 通常，FP树的大小比未压缩的数据小，因为购物篮数据的事务常常共享一些共同项
  - —在最好情况下，所有的事务都具有相同的项集，FP树只包含一条结点路径
  - —在最坏情况下，所有的事务都具有唯一项集，由于事务不包含任何共同项，FP树的大小实际上与原数据的大小一样，然而，由于需要附加的空间为每个项存放结点间的指针和计数，FP树的存储需求增大

# FP-Tree Construction

- **FP树的大小也取决于项的排序方式**
  - —如果颠倒前面例子的序，即项按照支持度由小到大排列

| TID | Items |
|---|---|
| 1 | {A,B} |
| 2 | {B,C,D} |
| 3 | {A,C,D,E} |
| 4 | {A,D,E} |
| 5 | {A,B,C} |
| 6 | {A,B,C,D} |
| 7 | {A} |
| 8 | {A,B,C} |
| 9 | {A,B,D} |
| 10 | {B,C,E} |

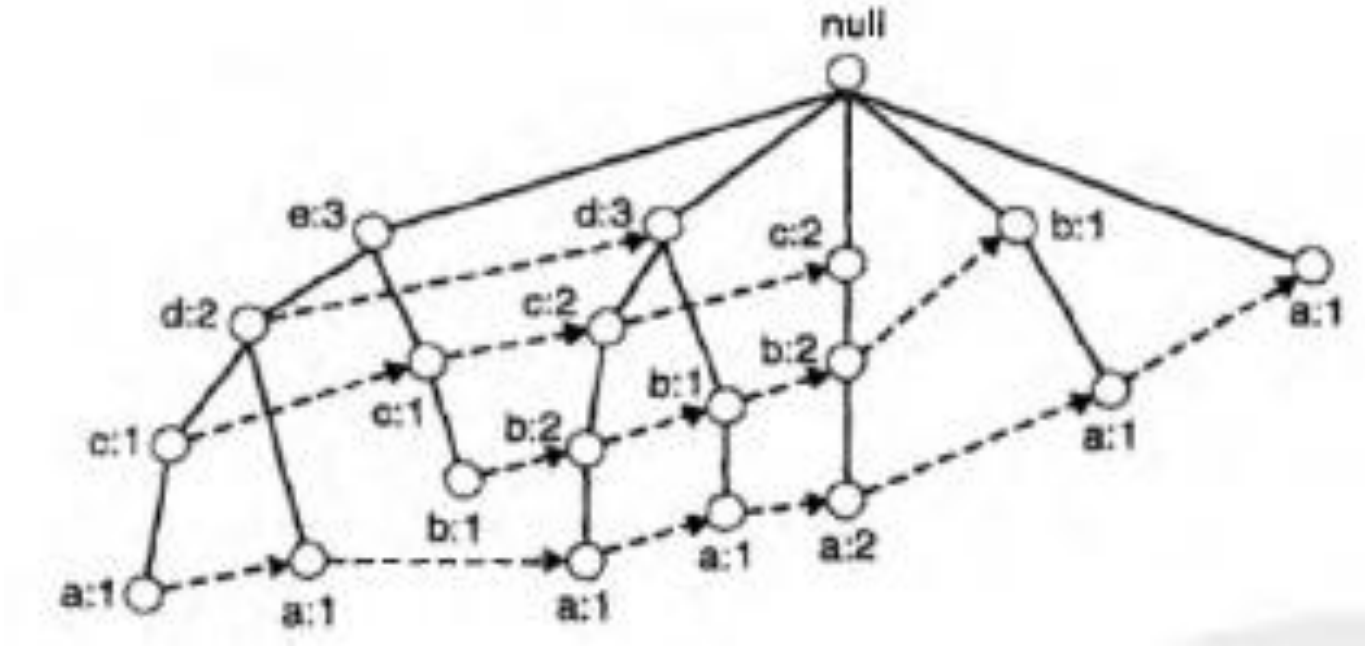


| Item | Count | Pointer |
|---|---|---|
| A | 8 | |
| B | 7 | |
| C | 6 | |
| D | 5 | |
| E | 3 | |



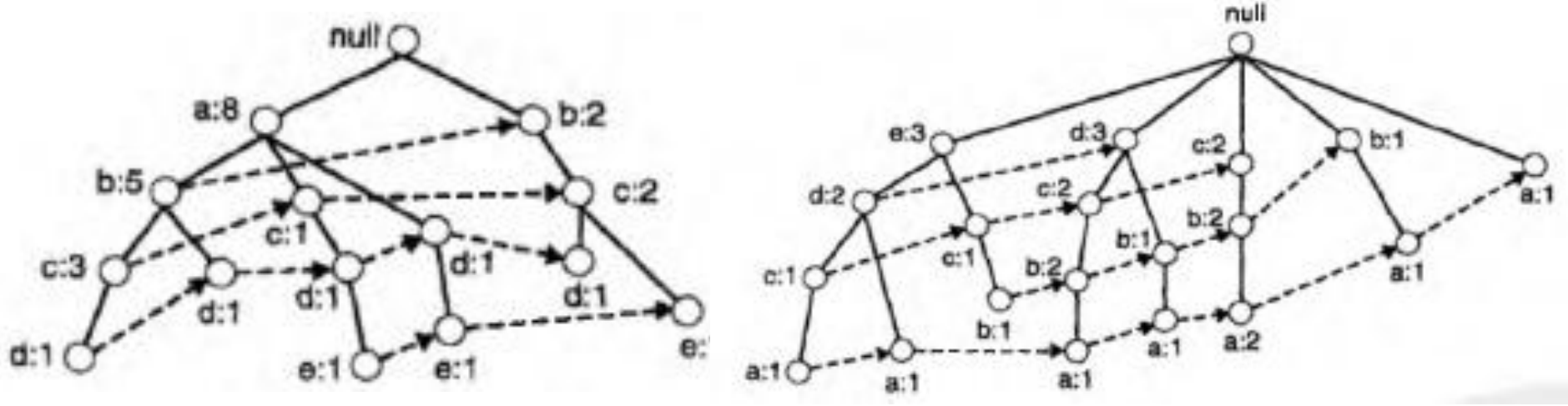
# FP-Tree Construction

- **FP树的大小也取决于项的排序方式**
  - —如果颠倒前面例子的序，即项按照支持度由小到大排列，则结果下图右所示的FP树会显得更加茂盛，因为根结点上的分支数由2增加到5，并且包含了高支持度项A和B的结点数由3 增加到12



# FP-Tree Construction

- FP树的大小也取决于项的排序方式
  - —尽管如此，支持度计数递减序并非总是导致最小的树。例如，假设加大给定的数据集，增加100个事务包含{E}、80个事务包含{D}、60个事务包含{C}、 40个事务包含{B} 。现在，项E是最频繁的，接下来依次是D、C、B、A，使用加大的事务数据，支持度计数递减序将导致类似下图右的树，而基于支持度计数递增序将产生一棵类似于下图左的树。



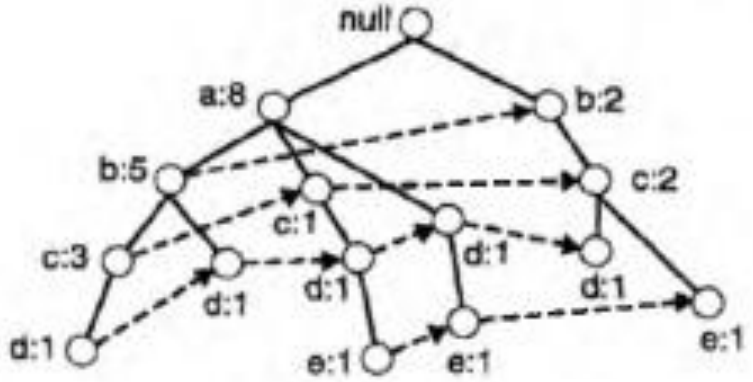
# FP-tree Structure的优势

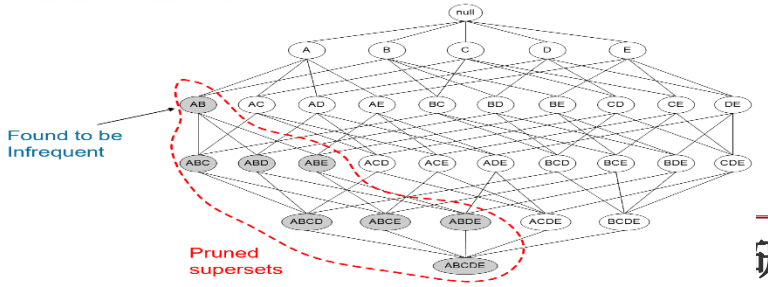
- 完整性
  - —保存频繁模式挖掘的完整的信息
  - —没有将任何事物破坏
- 紧凑
  - —项根据频率降序排列: 越频繁的结点越有可能被共享
  - —减少不相关信息—删除了非频繁项
  - —占用的不比原始数据量大 (不包括结点连接和计算时所占空间)

# FP增长算法的频繁项集产生原理

- FP增长是一种以自底向上方式探索树，由FP树产生频繁项集的算法
  - 给定上述FP树，算法首先查找以E结尾的频繁项集，接下来依次是D、C、B、A，这是用于发现以某一个特定项结尾的频繁项集的自底向上策略
  - 由于每一个事务都映射到FP树中的一条路径，因而通过仅考察包含特定结点（例如E）的路径，就可以发现以E结尾的频繁项集，使用与结点E相关联的指针，可以快速访问这些路径。

| Item | Count |
|---|---|
| A | 8 |
| B | 7 |
| C | 6 |
| D | 5 |
| E | 3 |



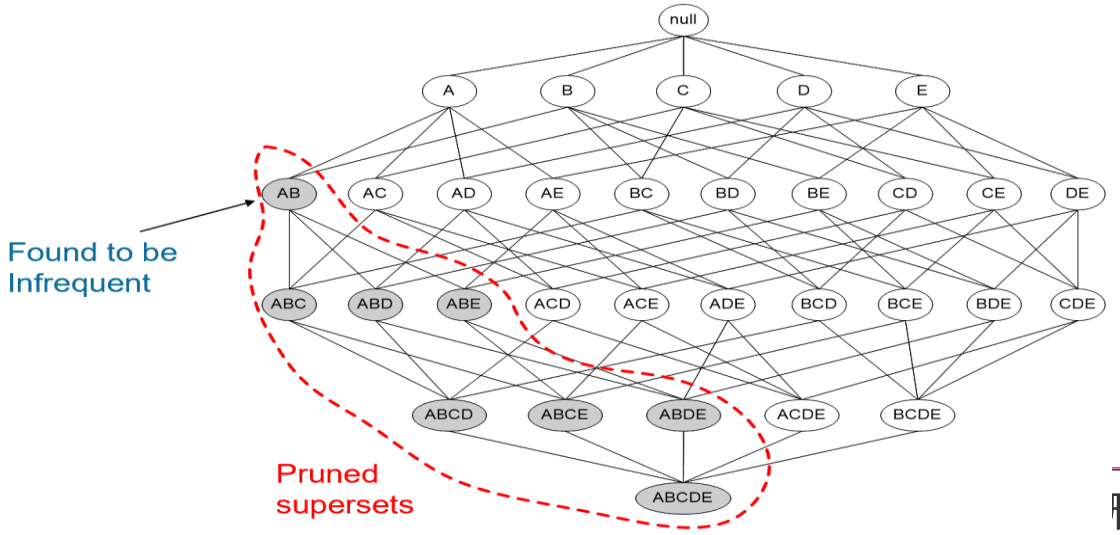



# FP增长算法的频繁项集产生原理

- FP增长采用分治策略将一个问题分解为较小的子问题，从而发现以某个特定后缀结尾的所有频繁项集

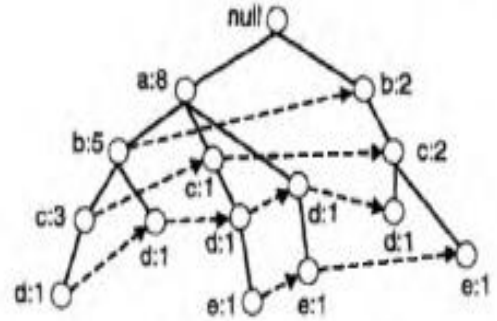
  —为发现所有以E结尾的频繁项集，必须首先检查项集{E}本身是否频繁。如果它是频繁的，则考虑发现以DE、CE、BE、AE结尾的频繁项集子问题

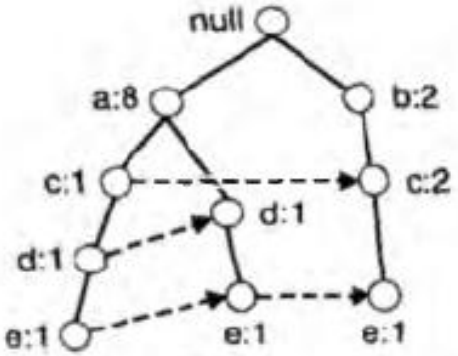
- 依次，每一个子问题可以进一步划分为更小的子问题
- 通过合并这些子问题得到的结果，就可以找到所有以E结尾的频繁项集



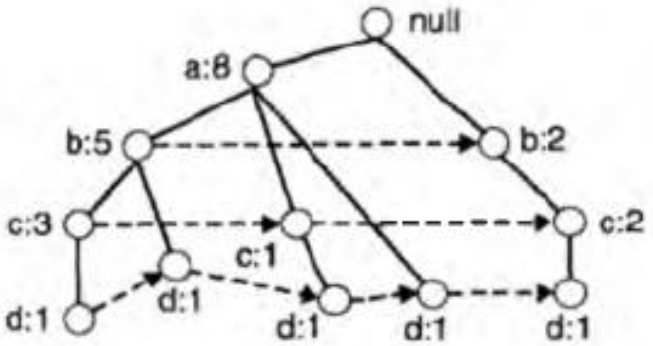
# FP增长算法的频繁项集产生原理

- 图a显示了所提取的包含节点E的路径，使用与结点E相关联的指针，可以快速访问这些路径。从中可以发现以E结尾的频繁项集
- 继续上述过程，依次找出以D、C、B、A结尾的频繁项集

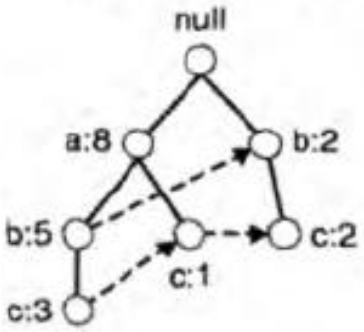


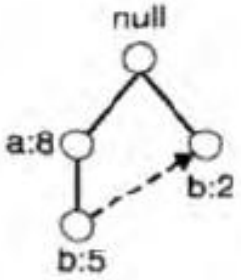

(a) 包含结点 $e$ 的路径




(b) 包含结点 $d$ 的路径




(c) 包含结点 $c$ 的路径




(d) 包含结点 $b$ 的路径




(e) 包含结点 $a$ 的路径

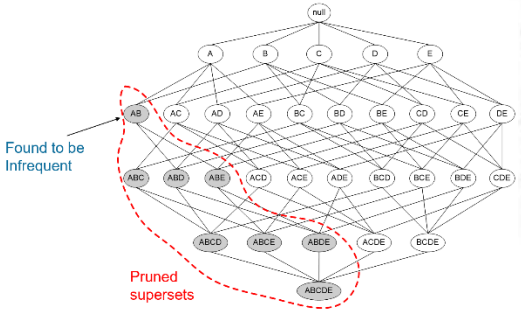



图 将频繁项集产生的问题分解成多个子问题，其中每个子问题分别涉及发现以 $e, d, c, b$ 和 $a$ 结尾的频繁项集

# FP增长算法的频繁项集产生
## ——发现以E结尾的频繁项集步骤

（1）收集包含E结点的所有路径，这些初始路径称为前缀路径

（2）由E的前缀路径，把与结点E相关联的支持度计数相加得到E的支持度计数是3，{E}是频繁项集（假定最小支持度为2 ）

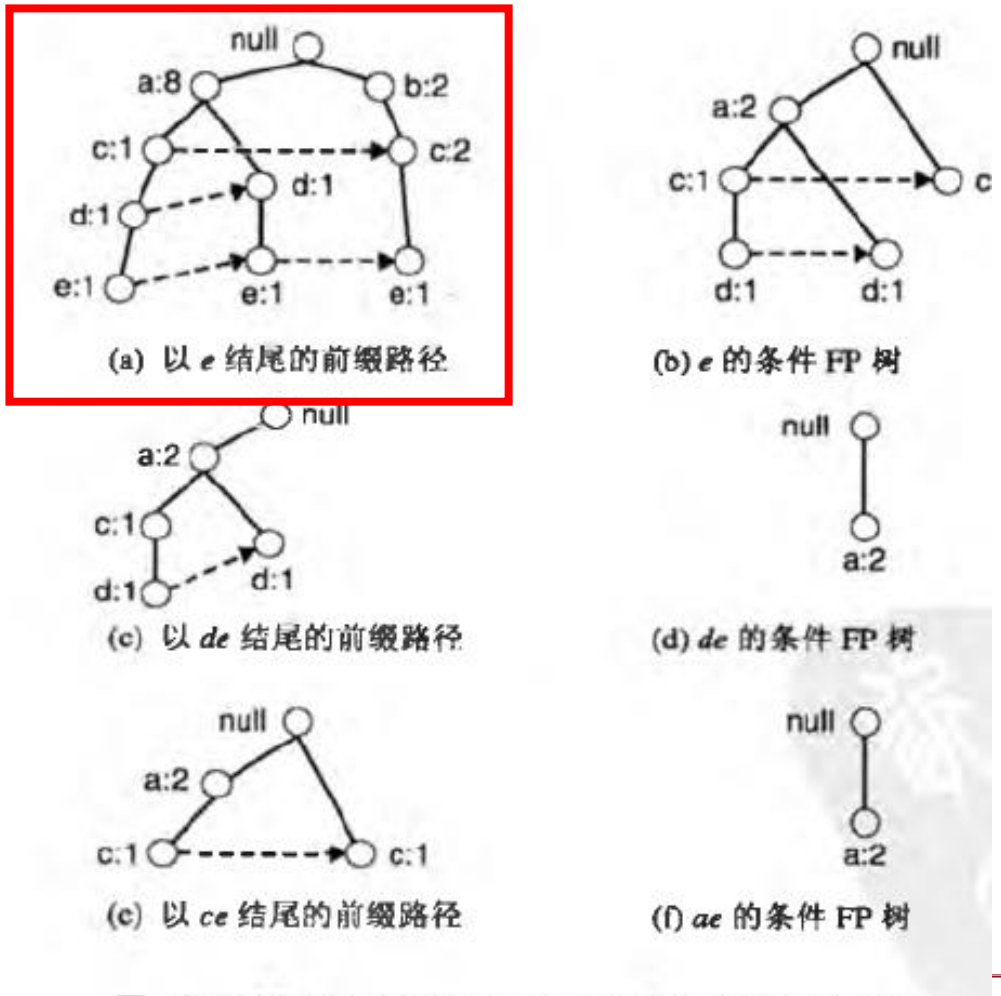



图 使用 FP 增长算法发现以 e 结尾的频繁项集的例子

# FP增长算法的频繁项集产生
## ——发现以E结尾的频繁项集步骤

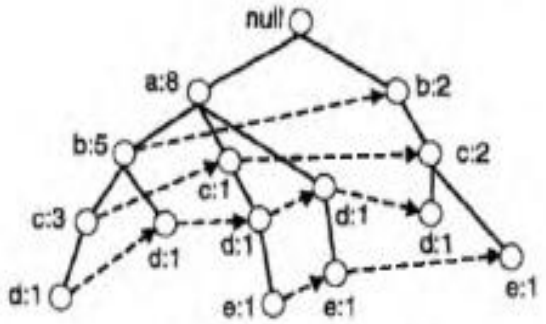
（3）由于E是频繁的，因此算法必须解决发现以DE、CE、BE、AE结尾的频繁项集的子问题。在解决这些子问题之前，必须先将前缀路径转化为条件FP树（conditional FP-tree），具体步骤如下：

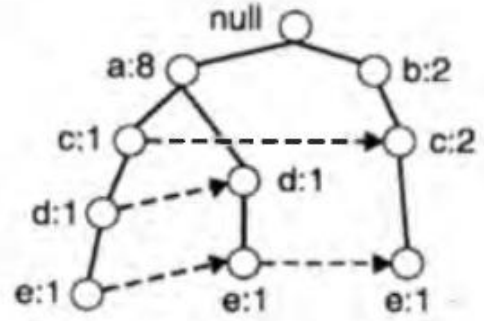
1）首先，必须更新前缀路径上的支持度计数，因为某些计数包括那些不含项E的事务。图(a)中B:2-C:2-E:1包括不含E的事务{B,C}，必须将该前缀路径上的计数调整为1，以反映包含{B,C,E} 事务实际个数

2）删除E结点，修剪前缀路径。删除E结点是因为，沿这些前缀路径的支持度计数已经更新，已反映包含E的那些事务，并且发现以DE、CE、BE、AE结尾的频繁项集的子问题不再需要结点E的信息

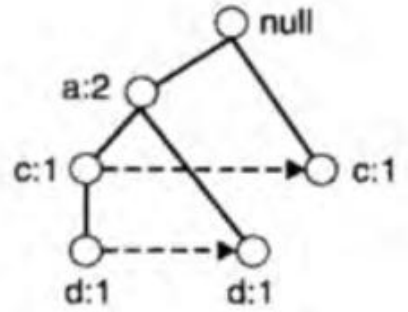
3）更新沿前缀路径上的支持度计数之后，某些项可能不再是频繁的。例如，结点B只出现 1次，它的支持度计数等于1，即只有一个事务同时包含B 和E，因而所有以BE结尾的项集一定都是非频繁的，所以在其后的分析中可以忽略B

4）E的条件FP树如图b，该树看上去与原来的前缀路径不同，因为频度计数已经更新，并且结点B和E已被删除



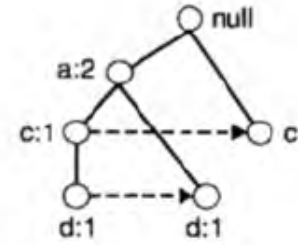

(a) 以 e 结尾的前缀路径



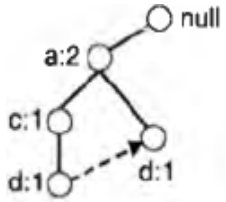

(b) e 的条件 FP 树

# FP增长算法的频繁项集产生
## ——发现以E结尾的频繁项集步骤

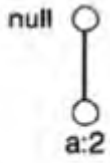
**（4）FP增长使用E的条件FP树来解决发现以DE、CE、BE、AE结尾的频繁项集的子问题**



(b) *e* 的条件 FP 树

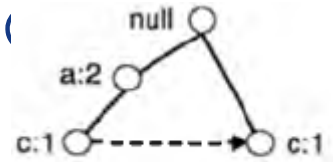


(c) 以 *de* 结尾的前缀路径

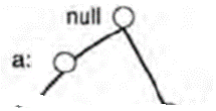


(d) *de* 的条件 FP 树

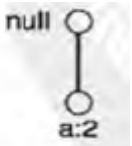
- 为了发现以DE结尾的频繁项集，从项E的条件树收集D的所有前缀路径（图c）
- 通过将与结点D相关联的频度计数求和，得到项集{D,E}的支持度计数。因为项集{D,E} 的支持度计数等于2，所以{D,E}是频繁项集。
- 接下来，算法采用第3 步介绍的方法构建DE的条件FP树（图d）。更新了支持度计数并删除了非频繁项C之后，DE的条件FP树显示在图d中。因为该条件FP树只包含一个支持度等于最小支持度的项A，算法提取出频繁项集{A,D,E}
- 算法转到下一个子问题，产生以CE结尾的频繁项集（图e） 。处理C的前缀路径后，只发现项集{C,E} 是频繁的
- 接下来，算法继续解决下一个子问题，发现项集{A,E} 是剩下唯一的频繁项集（



(e) 以 *ce* 结尾的前缀路径



Ce的条件FP树



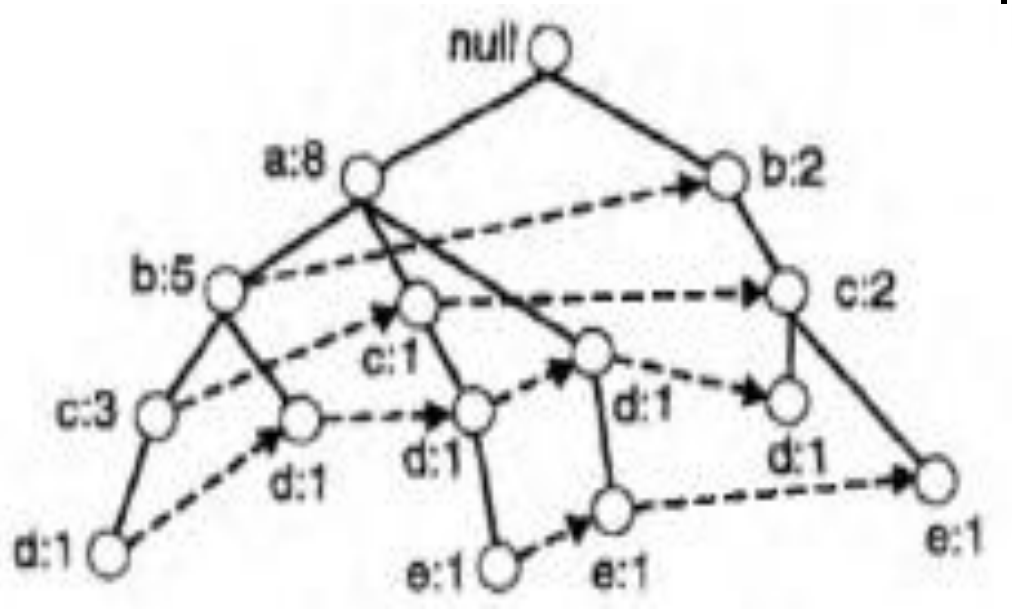
(f) *ae* 的条件 FP 树

| 后缀 | 频繁项集 |
|---|---|
| E | {E},{D,E},{A,D,E},{C,E},{A,E} |

# FP增长算法的频繁项集产生
## ——发现以D、C、B、A结尾的频繁项集步骤

- 随堂小作业

| 后缀 | 频繁项集 |
| --- | --- |
| E | {E},{D,E},{A,D,E},{C,E},{A,E} |
| D | ? |
| C | ? |
| B | ? |
| A | ? |



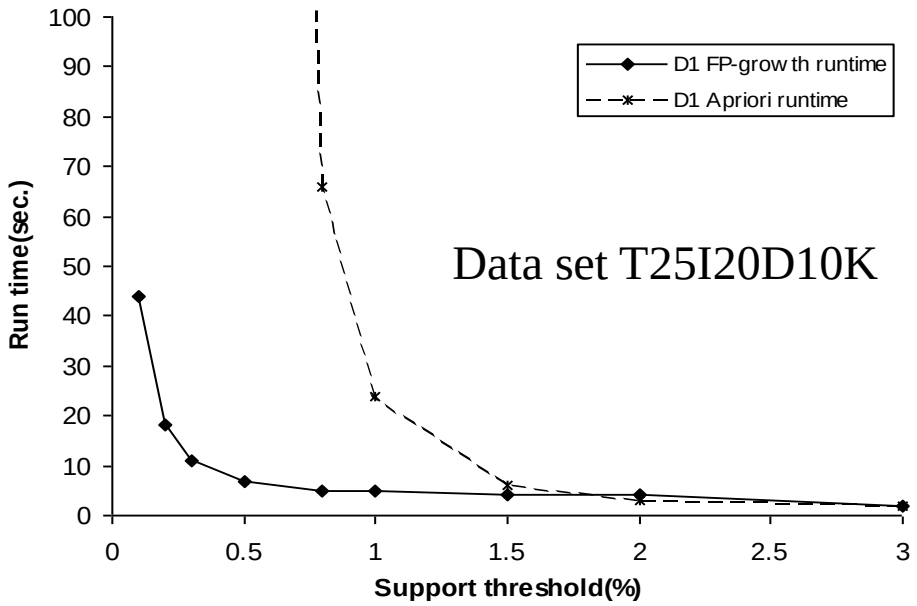
# 挖掘频繁项集的模式增长方法

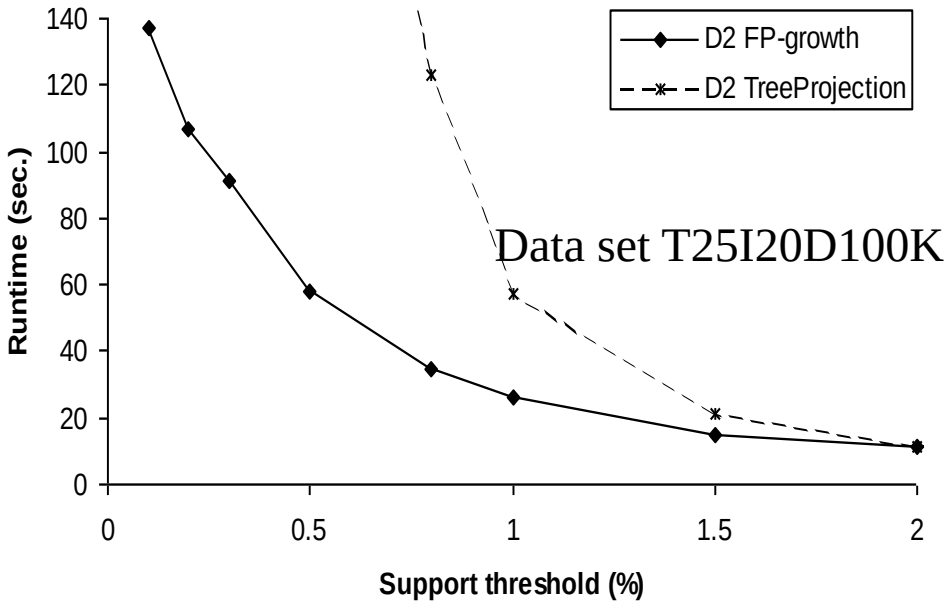
- 思想: 频繁模式增长
  - —递归地增长频繁模式
- 方法
  - —对于每个频繁项，构造条件模式基和条件FP树
  - —在每个新建的FP树上重复该过程
  - —直到最终的FP树为空，或它只有一条路径（该路径将生成所有子路径）

# 模式增长方法的优势

- 把一个大型的数据库压缩到一棵频繁模式树（FP-树）
  - —完整
    - 保存频繁模式挖掘的完整的信息
    - 没有将任何事物破坏
  - —紧凑
    - 项根据频率降序排列: 越频繁的结点越有可能被共享
    - 占用的不比原始数据量大 (不包括结点连接和计算时所占空间)，高浓缩，但对频繁项集挖掘是完整的
- 产生了一个高效的基于FP-树的频繁模式挖掘方法
  - —分治策略：把挖掘任务分解成小的任务
  - —避免候选集的产生，避免了高花销的数据库扫描：只检测子数据库
  - —减少不相关信息—删除了非频繁项

# Performance of FPGrowth in Large Datasets




**FP-Growth vs. Apriori**

FP-Growth vs. Tree-Projection

# 通过数据库投影扩展FP-Growth

- 如果内存中FP-tree放不下怎么办?
  - —数据库投影
    - 将一个数据库分成多个投影
    - 为每个投影构造FP-tree并挖掘
- Parallel projection vs. partition projection techniques
  - —Parallel projection
    - Project the DB in parallel for each frequent item
    - Parallel projection is space costly
    - All the partitions can be processed in parallel
  - —Partition projection
    - Partition the DB based on the ordered frequent items
    - Passing the unprocessed parts to the subsequent partitions

# Further Improvements of Mining Methods

- AFOPT (Liu, et al. @ KDD'03)
  - A "push-right" method for mining condensed frequent pattern (CFP) tree
- Carpenter (Pan, et al. @ KDD'03)
  - Mine data sets with small rows but numerous columns
  - Construct a row-enumeration tree for efficient mining
- FPgrowth+ (Grahne and Zhu, FIMI'03)
  - Efficiently Using Prefix-Trees in Mining Frequent Itemsets, Proc. ICDM'03 Int. Workshop on Frequent Itemset Mining Implementations (FIMI'03), Melbourne, FL, Nov. 2003
- TD-Close (Liu, et al, SDM'06)

# Extension of Pattern Growth Mining Methodology

- Mining closed frequent itemsets and max-patterns
  - CLOSET (DMKD'00), FPclose, and FPMax (Grahne & Zhu, Fimi'03)
- Mining sequential patterns
  - PrefixSpan (ICDE'01), CloSpan (SDM'03), BIDE (ICDE'04)
- Mining graph patterns
  - gSpan (ICDM'02), CloseGraph (KDD'03)
- Constraint-based mining of frequent patterns
  - Convertible constraints (ICDE'01), gPrune (PAKDD'03)
- Computing iceberg data cubes with complex measures
  - H-tree, H-cubing, and Star-cubing (SIGMOD'01, VLDB'03)
- Pattern-growth-based Clustering
  - MaPle (Pei, et al., ICDM'03)
- Pattern-Growth-Based Classification
  - Mining frequent and discriminative patterns (Cheng, et al, ICDE'07)

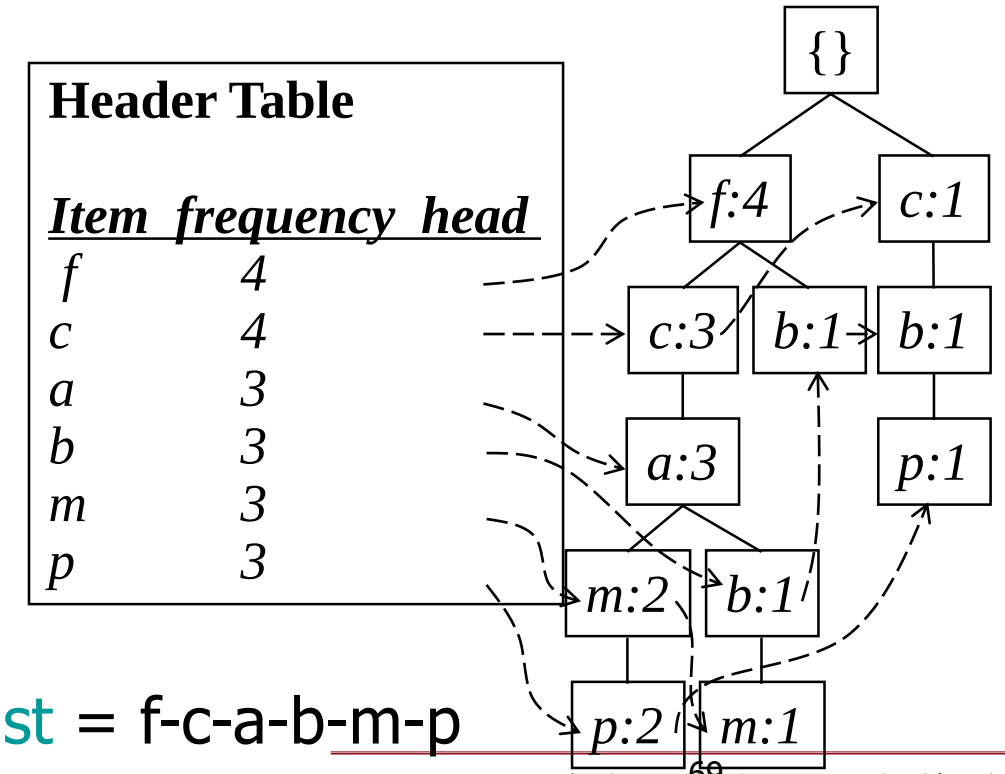
# 教材中关于FPGrowth方法的解释

# 从事务数据库构造FP-tree

| TID | Items bought | (ordered) frequent items |
|---|---|---|
| 100 | {f, a, c, d, g, i, m, p} | {f, c, a, m, p} |
| 200 | {a, b, c, f, l, m, o} | {f, c, a, b, m} |
| 300 | {b, f, h, j, o, w} | {f, b} |
| 400 | {b, c, k, s, p} | {c, b, p} |
| 500 | {a, f, c, e, l, p, m, n} | {f, c, a, m, p} |

*min_support = 3*

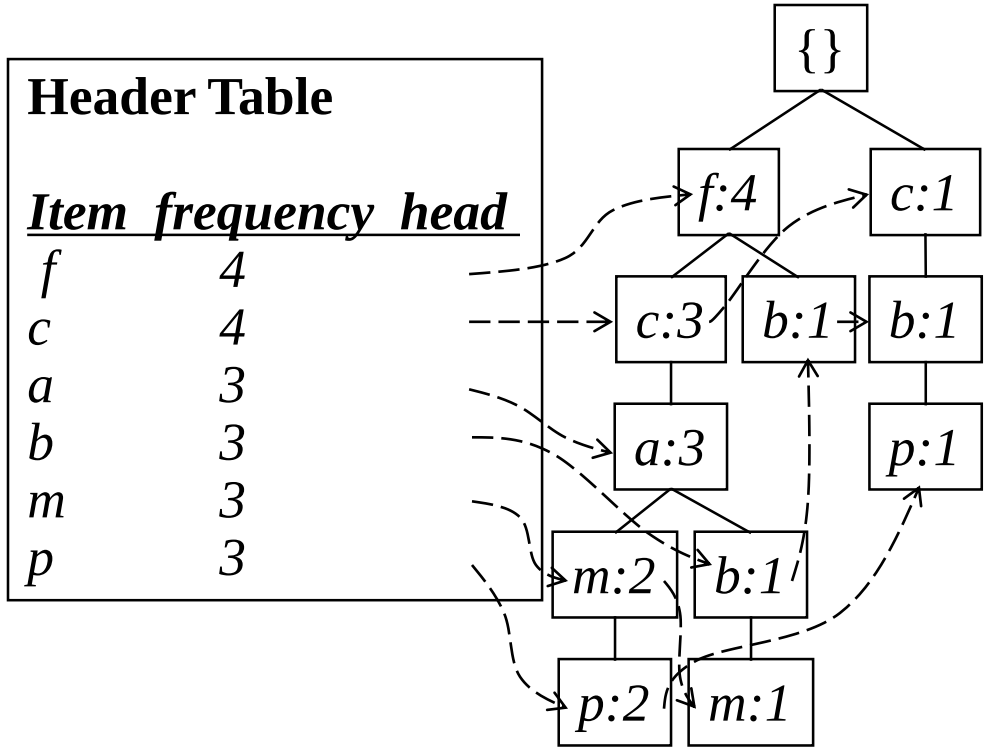
1. 扫描一次数据库，找到长度为1的频繁的项
2. 将频繁项根据支持度降序排列
3. 再次扫描数据库，构造FP-tree




F-list = f-c-a-b-m-p

# 在P-conditional Database发现蕴含p的模式

- 由FP-tree的head结点开始向下遍历到p
- 沿着每条含p的连接开始横着在FP-tree中寻找最后结点
- 找到所有到p的前缀路径，以形成p的条件模式基

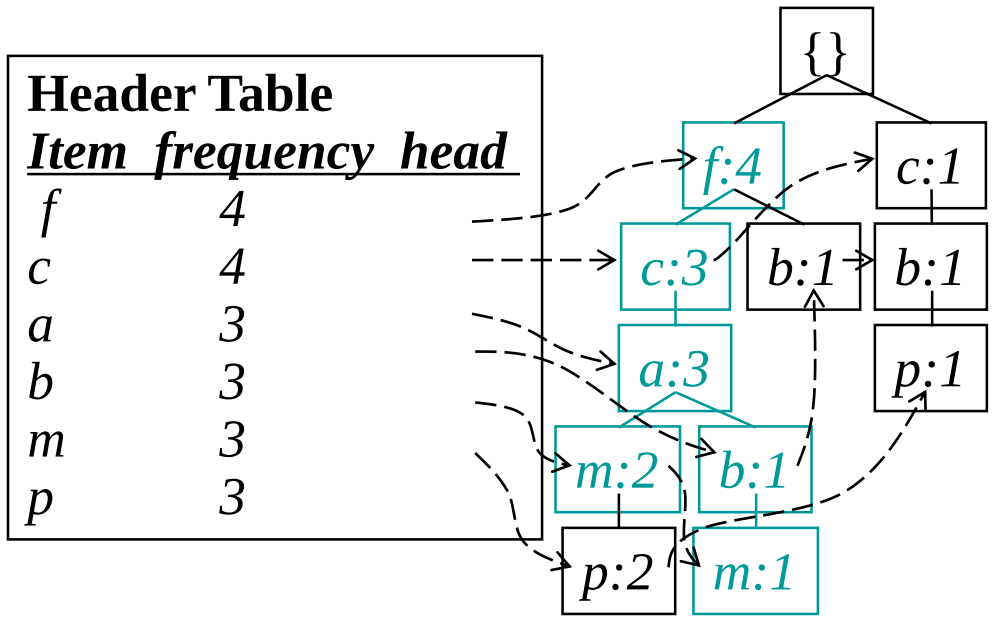



***Conditional* pattern bases**

| *item* | *cond. pattern base* |
|---|---|
| *c* | *f:3* |
| *a* | *fc:3* |
| *b* | *fca:1, f:1, c:1* |
| *m* | *fca:2, fcab:1* |
| *p* | *fcam:2, cb:1* |

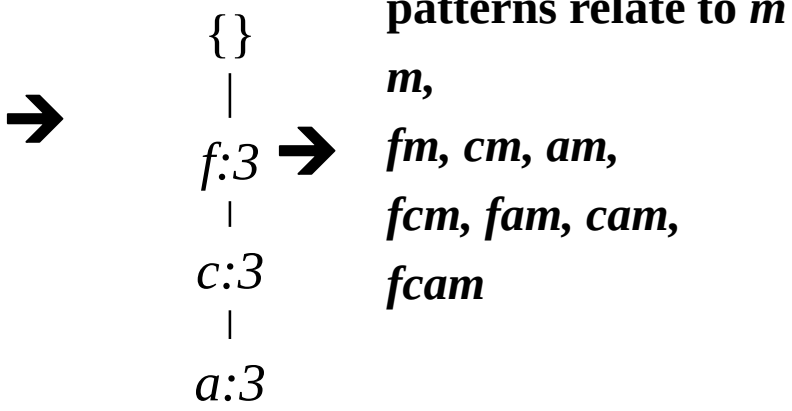
# 由条件模式基生成条件 FP-trees

- 对于每个条件模式基
  - —为模式基中的每个项计数
  - —为模式基中的频繁项构建FP-tree




***m-conditional* pattern base:**

***fca:2, fcab:1***




***m-conditional* FP-tree**

**All frequent patterns relate to *m***

***m*,**

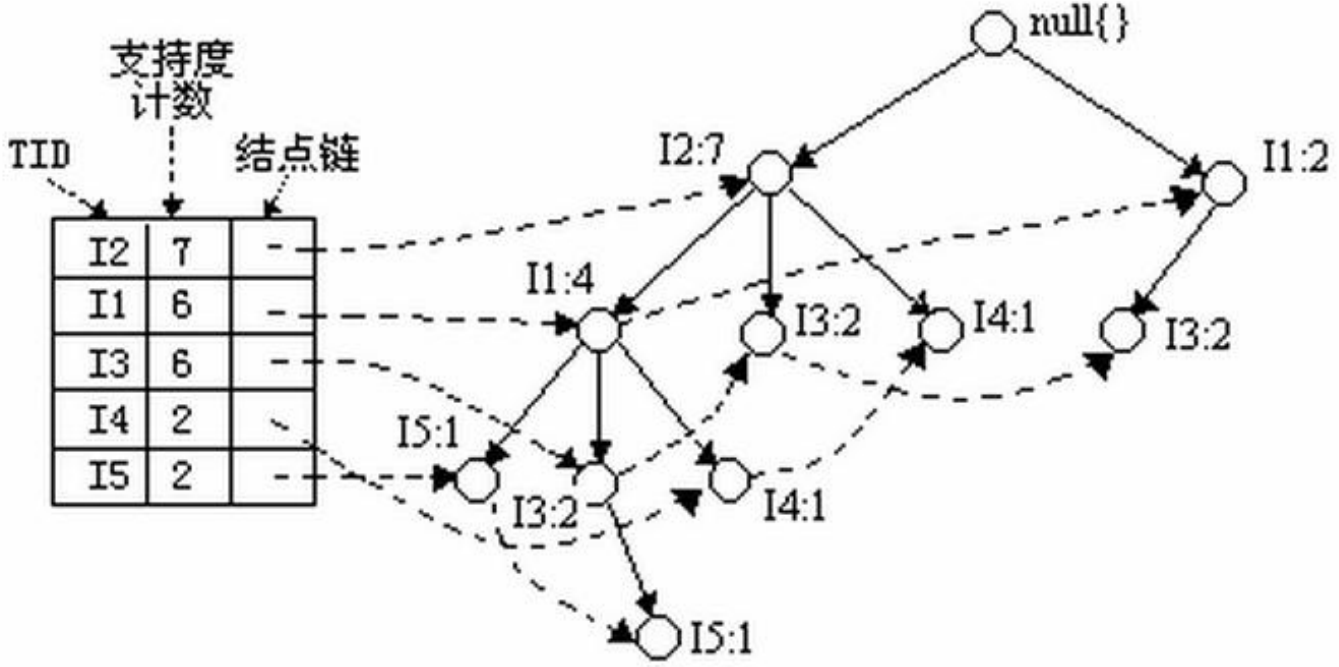
***fm, cm, am,***

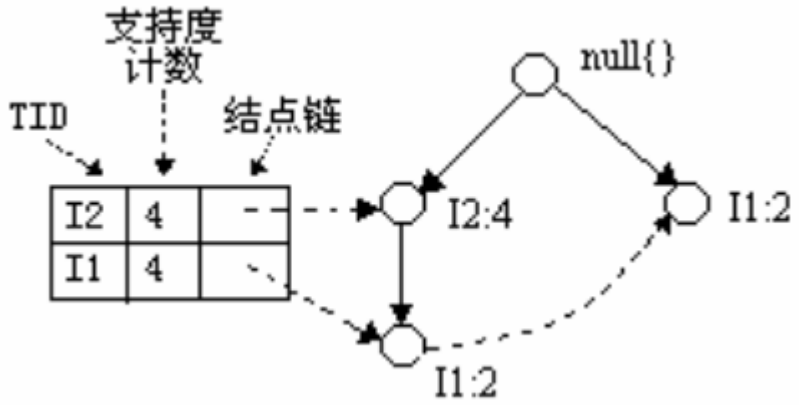
***fcm, fam, cam,***

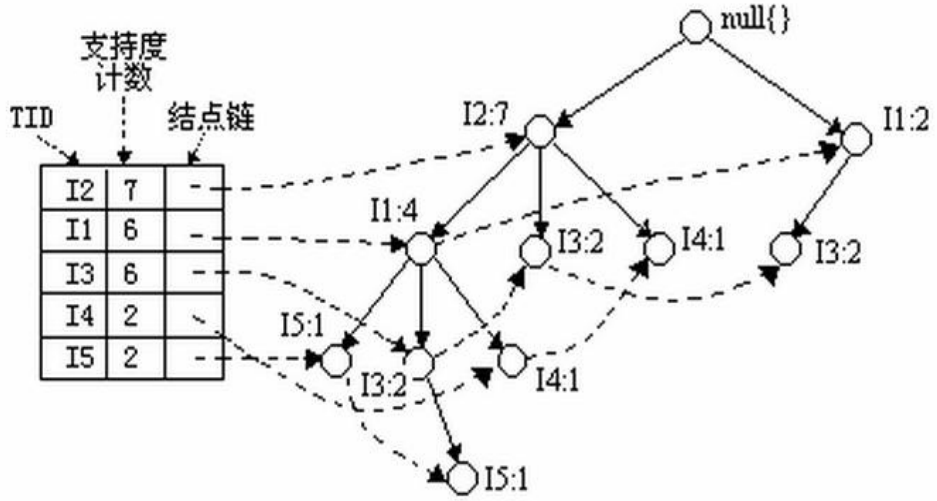
***fcam***

**AllElectronics 数据库**

| TID | List of item_ID's |
|---|---|
| **T100** | I1,I2,I5 |
| T200 | I2,I4 |
| T300 | I2,I3 |
| T400 | I1,I2,I4 |
| T500 | I1,I3 |
| T600 | I2,I3 |
| T700 | I1,I3 |
| T800 | I1,I2,I3,I5 |
| T900 | I1,I2,I3 |






| 项 | 条件模式基 | 条件 FP 树 | 产生的频繁模式 |
|---|---|---|---|
| I5 | {{I2, I1: 1}, {I2, I1, I3: 1}} | ⟨I2: 2, I1: 2⟩ | {I2, I5: 2}、{I1, I5: 2}、{I2, I1, I5: 2} |
| I4 | {{I2, I1: 1}, {I2: 1}} | ⟨I2: 2⟩ | {I2, I4: 2} |
| I3 | {{I2, I1: 2}, {I2: 2}, {I1: 2}} | ⟨I2: 4, I1: 2⟩, ⟨I1: 2⟩ | {I2, I3: 4}、{I1, I3: 4}、{I2, I1, I3: 2} |
| I1 | {{I2: 4}} | ⟨I2: 4⟩ | {I2, I1: 4} |

# FP Growth算法

**算法：FP-Growth。使用 FP 树，通过模式增长挖掘频繁模式。**

**输入：**最小化

- *D*：事务数据库。
- *min_ sup*：最小支持度阈值。

**输出：**频繁模式的完全集。

**方法：**

1. 按以下步骤构造 FP 树：
   (a) 扫描事务数据库 *D* 一次。收集频繁项的集合 *F* 和它们的支持度计数。对 *F* 按支持度计数降序排序，结果为频繁项列表 *L*。
   (b) 创建 FP 树的根结点，以“null”标记它。对于 *D* 中每个事务 *Trans*，执行：
   选择 *Trans* 中的频繁项，并按 *L* 中的次序排序。设 *Trans* 排序后的频繁项列表为 $[p \mid P]$，其中 $p$ 是第一个元素，而 $P$ 是剩余元素的列表。调用 **insert_tree**$([p \mid P], T)$。该过程执行情况如下。如果 *T* 有子女 *N* 使得 *N.item-name* = *p.item-name*，则 *N* 的计数增加 1；否则，创建一个新结点 *N*，将其计数设置为 1，链接到它的父结点 *T*，并且通过结点链结构将其链接到具有相同 *item-name* 的结点。如果 $P$ 非空，则递归地调用 **insert_tree**$(P, N)$。
2. FP 树的挖掘通过调用 **FP_growth**(*FP_tree*, *null*) 实现。该过程实现如下。

**procedure FP_growth**(*Tree*, $\alpha$)
(1) **if** *Tree* 包含单个路径 *P* **then**
(2) **for** 路径 *P* 中结点的每个组合（记作 $\beta$）
(3) 产生模式 $\beta \cup \alpha$，其支持度计数 *support_count* 等于 $\beta$ 中结点的最小支持度计数；
(4) **else for** *Tree* 的头表中的每个 $a_i$
(5) 产生一个模式 $\beta = a_i \cup \alpha$，其支持度计数 *support_count* = $a_i$.*support_count*；
(6) 构造 $\beta$ 的条件模式基，然后构造 $\beta$ 的条件 *FP* 树 $Tree_\beta$；
(7) **if** $Tree_\beta \neq \varnothing$ **then**
(8) 调用 **FP_growth**($Tree_\beta$, $\beta$)；

# 可扩展的频繁项集挖掘方法

- Apriori:候选生成和测试方法
- 提升Apriori的效率
- FPGrowth:频繁模式增长的方法
- ECLAT:垂直数据格式下的频繁模式挖掘

# ECLAT: 使用垂直数据格式挖掘频繁项集

- 水平数据格式

- 垂直数据格式

AllElectronics 数据库

| TID | List of item_ID's |
|---|---|
| **T100** | I1,I2,I5 |
| T200 | I2,I4 |
| T300 | I2,I3 |
| T400 | I1,I2,I4 |
| T500 | I1,I3 |
| T600 | I2,I3 |
| T700 | I1,I3 |
| T800 | I1,I2,I3,I5 |
| T900 | I1,I2,I3 |

**Table 6.3** The Vertical Data Format of the Transaction Data Set *D* of Table 6.1

| itemset | TID_set |
|---|---|
| I1 | {T100, T400, T500, T700, T800, T900} |
| I2 | {T100, T200, T300, T400, T600, T800, T900} |
| I3 | {T300, T500, T600, T700, T800, T900} |
| I4 | {T200, T400} |
| I5 | {T100, T800} |

表 6.1　AllElectronics 某分店的事务数据

| *TID* | 商品 *ID* 的列表 | *TID* | 商品 *ID* 的列表 |
|---|---|---|---|
| T100 | I1，I2，I5 | T600 | I2，I3 |
| T200 | I2，I4 | T700 | I1，I3 |
| T300 | I2，I3 | T800 | I1，I2，I3，I5 |
| T400 | I1，I2，I4 | T900 | I1，I2，I3 |
| T500 | I1，I3 | | |

最小支持度为2

表 6.3　表 6.1 事务数据库 *D* 的垂直数据格式

| 项集 | TID－集 | 项集 | TID－集 |
|---|---|---|---|
| I1 | {T100，T400，T500，T700，T800，T900} | I4 | {T200，T400} |
| I2 | {T100，T200，T300，T400，T600，T800，T900} | I5 | {T100，T800} |
| I3 | {T300，T500，T600，T700，T800，T900} | | |

表 6.4　垂直数据格式的 2 项集

| 项集 | *TID*－集 | 项集 | *TID*－集 |
|---|---|---|---|
| {I1，I2} | {T100，T400，T800，T900} | {I2，I3} | {T300，T600，T800，T900} |
| {I1，I3} | {T500，T700，T800，T900} | {I2，I4} | {T200，T400} |
| {I1，I4} | {T400} | {I2，I5} | {T100，T800} |
| {I1，I5} | {T100，T800} | {I3，I5} | {T800} |

表 6.5　垂直数据格式的 3 项集

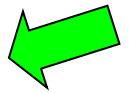
| 项集 | *TID*－集 |
|---|---|
| {I1，I2，I3} | {T800，T900} |
| {I1，I2，I5} | {T100，T800} |

# 算法效率分析

- 除了在产生候选（k + 1）项集时利用先验性质外，这种方法的另一优点是不需要扫描数据库来确定（k + 1）项集的支持度（k+1）。这是因为每个k项集的集携带了计算支持度的完整信息

- 然而，TID集可能很长，需要大量内存空间，长集合的交运算还需要大量的计算时间

# Chapter 6
# 挖掘频繁模式、关联和相关性基本概念和方法

- 基本概念
- 频繁项集挖掘方法
- 哪些模式是有趣的：模式评估方法 
- 小结

# 关联规则的评估

| 单项统计 | 支持度 |
|---|---|
| {啤酒} | 2 |
| {烤鸭} | 3 |
| {面饼} | 3 |
| {面酱} | 4 |

- 5条事务记录
- 单项统计，60%顾客买了烤鸭、60%顾客买了面酱
- 双项统计，60%的 顾客买了烤鸭和面酱
- 三项统计，40%的顾客同时买了烤鸭、面饼和面酱

| 双项统计 | 支持度 |
|---|---|
| {啤酒，面饼} | 2 |
| {烤鸭，面饼} | 2 |
| {烤鸭，面酱} | 3 |
| {面饼，面酱} | 2 |

| 三项统计 | 支持度 |
|---|---|
| {烤鸭，面饼，面 | 2 |

- R1：烤鸭➔面饼、面酱。支持度40%，置信度为66.6%
- R2：面饼➔烤鸭、面酱。支持度40%，置信度为66.6%
- R3：面酱➔面饼、烤鸭。支持度40%，置信度为50%
- KDD结果不一定是因果关系。运用之妙成乎于人。
- 例如：
- 用R1，将烤鸭降价以促销面饼、面酱，很可能会破产
- 用R2　将面饼降价，以促销烤鸭，可能会发财；
- 用R3，引不起顾客的热情。

# 强规则不一定是有趣的

- 假设对分析涉及购买计算机游戏和录像的AllElectronics的事务感兴趣
- 设game表示包含计算机游戏的事务，而video表示包含录像的事务。在所分析的10000个事务中
  - 6000个顾客事务包含计算机游戏
  - 7500个事务包含录像
  - 4000个事务间时包含计算机游戏和录像
- 假设发现关联规则的数据挖掘程序在该数据上运行，使用最小支持度30%，最小置信度60%

$$buys(X, \text{“}computer\ games\text{”}) \Rightarrow buys(X, \text{“}videos\text{”})$$

$$[support = 40\%, confidence = 66\%]$$

- 虽然满足最小支持度和最小置信度阈值，然而规则是误导，因为购买录像的概率是75%比66%还高。事实上，计算机游戏和录像是负相关的，因为买一种实际上降低购买另一种的可能性。不完全理解这种现象，容易根据规则做出不明智的商务决定
- 表明规则的置信度有一定的欺骗性，它并不度量A与B之间相关和蕴涵的实际强度（或缺乏强度），寻求支持度—置信度框架的替代，对挖掘有趣的数据联系可能是有用的

# 从关联分析到相关分析

- **使用相关性度量来扩充关联规则的支持度- 置信度框架**
  - —相关规则不仅用支持度和置信度度量，而且还用项集A和B之间的相关性度量：**相关规则**[ˌkɔːrəˈleɪʃn]

$$A \Rightarrow B\,[\,support, confidence, correlation\,]$$

- **有许多不同的相关性度量可供选择**

# 第一种相关性度量：提升度lift

Cereal 谷物
[ˈsɪəriəl]

- ***play basketball*** ⇒ ***eat cereal*** **[40%, 66.7%] is misleading**
  - **The overall % of students eating cereal is 75% > 66.7%.**
- ***play basketball*** ⇒ ***not eat cereal*** **[20%, 33.3%] is more accurate, although with lower support and confidence**
- **Measure of dependent/correlated events: lift**

$$lift = \frac{P(A \cup B)}{P(A)P(B)}$$

| | Basketball | Not basketball | Sum (row) |
|---|---|---|---|
| Cereal | 2000 | 1750 | 3750 |
| Not cereal | 1000 | 250 | 1250 |
| Sum(col.) | 3000 | 2000 | 5000 |

$$lift(B,C) = \frac{2000/5000}{3000/5000 * 3750/5000} = 0.89$$

$$lift(B,\neg C) = \frac{1000/5000}{3000/5000 * 1250/5000} = 1.33$$

# 第二种相关性度量：卡方计算

| | Play chess | Not play chess | Sum (row) |
|---|---|---|---|
| Like science fiction | 250(90) | 200(360) | 450 |
| Not like science fiction | 50(210) | 1000(840) | 1050 |
| Sum(col.) | 300 | 1200 | 1500 |

$$e_{11} = \frac{count(\text{Play chess}) \times count(\text{Like science fiction})}{n} = \frac{300 \times 450}{1500} = 90$$

- $X^2$ (卡方)卡方计算(括号中的数字是根据两类的数据分布预期的)

$$\chi^2 = \frac{(250-90)^2}{90} + \frac{(50-210)^2}{210} + \frac{(200-360)^2}{360} + \frac{(1000-840)^2}{840} = 507.93$$

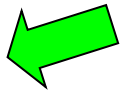
- 对于自由度1，在0.001的置信水平下，拒绝假设的值是10.828（取自$X^2$ 分布上百分点表，参见统计学教科书），由于计算的值大于该值，因此我们可以拒绝Play chess与Like science fiction是独立的假设，并断言对于给定的人群，这两个属性是（强）相关的。

# 提升度和 $\chi^2$ 是好的检测相关性的方法吗?

- 支持度和置信度不能说明相关性
- 超过20种度量兴趣度的方法被提出 (see Tan, Kumar, Sritastava @KDD'02)
- Which are good ones?

| symbol | measure | range | formula |
|---|---|---|---|
| $\phi$ | $\phi$-coefficient | -1 . . . 1 | $\frac{P(A,B)-P(A)P(B)}{\sqrt{P(A)P(B)(1-P(A))(1-P(B))}}$ |
| $Q$ | Yule's Q | -1 . . . 1 | $\frac{P(A,B)P(\overline{A},\overline{B})-P(A,\overline{B})P(\overline{A},B)}{P(A,B)P(\overline{A},\overline{B})+P(A,\overline{B})P(\overline{A},B)}$ |
| $Y$ | Yule's Y | -1 . . . 1 | $\frac{\sqrt{P(A,B)P(\overline{A},\overline{B})}-\sqrt{P(A,\overline{B})P(\overline{A},B)}}{\sqrt{P(A,B)P(\overline{A},\overline{B})}+\sqrt{P(A,\overline{B})P(\overline{A},B)}}$ |
| $k$ | Cohen's | -1 . . . 1 | $\frac{P(A,B)+P(\overline{A},\overline{B})-P(A)P(B)-P(\overline{A})P(\overline{B})}{1-P(A)P(B)-P(\overline{A})P(\overline{B})}$ |
| $PS$ | Piatetsky-Shapiro's | -0.25 . . . 0.25 | $P(A,B)-P(A)P(B)$ |
| $F$ | Certainty factor | -1 . . . 1 | $\max(\frac{P(B\|A)-P(B)}{1-P(B)}, \frac{P(A\|B)-P(A)}{1-P(A)})$ |
| $AV$ | added value | -0.5 . . . 1 | $\max(P(B\|A)-P(B), P(A\|B)-P(A))$ |
| $K$ | Klosgen's Q | -0.33 . . . 0.38 | $\sqrt{P(A,B)}\max(P(B\|A)-P(B), P(A\|B)-P(A))$ |
| $g$ | Goodman-kruskal's | 0 . . . 1 | $\frac{\Sigma_j \max_k P(A_j,B_k)+\Sigma_k \max_j P(A_j,B_k)-\max_j P(A_j)-\max_k P(B_k)}{2-\max_j P(A_j)-max_k P(B_k)}$ |
| $M$ | Mutual Information | 0 . . . 1 | $\frac{\Sigma_i\Sigma_j P(A_i,B_j)\log\frac{P(A_i,B_j)}{P(A_i)P(B_j)}}{\min(-\Sigma_i P(A_i)\log P(A_i), -\Sigma_i P(B_i)\log P(B_i))}$ |
| $J$ | J-Measure | 0 . . . 1 | $\max(P(A,B)\log(\frac{P(B\|A)}{P(B)})+P(A\overline{B})\log(\frac{P(\overline{B}\|A)}{P(\overline{B})}))$ |
| $G$ | Gini index | 0 . . . 1 | $P(A,B)\log(\frac{P(A\|B)}{P(A)})+P(A\overline{B})\log(\frac{P(A\|\overline{B})}{P(A)})$ $\max(P(A)[P(B\|A)^2+P(\overline{B}\|A)^2]+P(\overline{A}[P(B\|\overline{A})^2+P(\overline{B}\|\overline{A})^2]-P(B)^2-P(\overline{B})^2,$ $P(B)[P(A\|B)^2+P(\overline{A}\|B)^2]+P(\overline{B}[P(A\|\overline{B})^2+P(\overline{A}\|\overline{B})^2]-P(A)^2-P(\overline{A})^2)$ |
| $s$ | support | 0 . . . 1 | $P(A,B)$ |
| $c$ | confidence | 0 . . . 1 | $max(P(B\|A), P(A\|B))$ |
| $L$ | Laplace | 0 . . . 1 | $\max(\frac{NP(A,B)+1}{NP(A)+2}, \frac{NP(A,B)+1}{NP(B)+2})$ |
| $IS$ | Cosine | 0 . . . 1 | $\frac{P(A,B)}{\sqrt{P(A)P(B)}}$ |
| $\gamma$ | coherence(Jaccard) | 0 . . . 1 | $\frac{P(A,B)}{P(A)+P(B)-P(A,B)}$ |
| $\alpha$ | all_confidence | 0 . . . 1 | $\frac{P(A,B)}{\max(P(A),P(B))}$ |
| $o$ | odds ratio | 0 . . . $\infty$ | $\frac{P(A,B)P(\overline{A},\overline{B})}{P(\overline{A},B)P(A,\overline{B})}$ |
| $V$ | Conviction | 0.5 . . . $\infty$ | $\max(\frac{P(A)P(\overline{B})}{P(A\overline{B})}, \frac{P(B)P(\overline{A})}{P(B\overline{A})})$ |
| $\lambda$ | lift | 0 . . . $\infty$ | $\frac{P(A,B)}{P(A)P(B)}$ |
| $S$ | Collective strength | 0 . . . $\infty$ | $\frac{P(A,B)+P(\overline{AB})}{P(A)P(B)+P(\overline{A})P(\overline{B})}\times\frac{1-P(A)P(B)-P(\overline{A})P(\overline{B})}{1-P(A,B)-P(\overline{AB})}$ |
| $\chi^2$ | $\chi^2$ | 0 . . . $\infty$ | $\Sigma_i\frac{(P(A_i)-E_i)^2}{E_i}$ |

# Chapter 6
# 挖掘频繁模式、关联和相关性基本概念和方法

- 基本概念
- 频繁项集挖掘方法
- 哪些模式是有趣的：模式评估方法
- 小结 

# Summary

- Basic concepts: association rules, support-confident framework
- Scalable frequent pattern mining methods
  - Apriori (Candidate generation & test)
  - Projection-based (FPgrowth, CLOSET+, ...)
  - Vertical format approach (ECLAT, CHARM, ...)
- Which patterns are interesting?
  - Pattern evaluation methods